

1 Introduction

1.1 Computer Vision Applications

Computer vision is used for many day-to-day things, such as:

- **Entertainment/media** – i.e. Searching digital image archives, automated context retrieval systems-CBIR; 3D Digital TV, cinematography, digital studios, and special effects; 3D capture of virtual humans and spaces; Gesture based input for games, e.g. Kinect or Wii.
- **Medicine and Medical Imaging** – i.e. Automatic Screening of X-rays, mammograms, MRI, PET, etc; 3D clinical photography for surface anatomy assessment; surgical planning systems.
- **Security & Surveillance** – i.e. Surveillance for intruder detection and vehicle monitoring; Driver attention/alertness monitoring & hazard detection.
- **Environment and Remote Monitoring** – i.e. Automated map-making and GIS revision; Telepresence (NASA mars explorer); Mobile phone camera apps (location and object recognition).
- **Robotics, Automation & Quality Control** – i.e. Spatial localisation and mapping of the environment; Autonomous vehicle navigation (driverless cars); Object recognition and scene understanding.

1.2 Digital Images

To process an image with computers we need to digitise it – we use a function $f(x, y)$ where x, y are spatial coordinates and f is a function of the intensity of the image at every point. An image has a finite number of pixels, it is common to quantise pixels in 8 bits [0..255]. It is always a positive number for display.

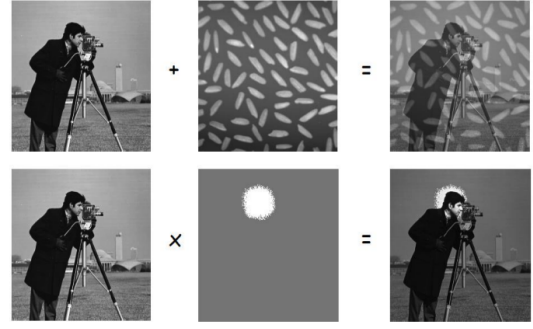
Pixels are stored and organised into a 2D array, indexed $[R, C]$ $N, M \in \mathbb{I}$ (integers). Human day-vision (called photopic vision) is based on cone photoreceptors in the retina. There are 3 types of cone tuned to red, green, and blue light respectively. Each type contains red, green, and blue pigment (rhodopsin).

Colour cameras filter light into red, green, and blue components. These image components are digitised into three colour panes representing red, green, and blue. Each pixel now comprises 3 numbers, representing red, green, and blue.

1.2.1 Image Arithmetic

Arbitrary numeric manipulation, e.g. $\{+, -, \times, \div\}$, with a constant applied to an image array, or images operating on images (element by element arithmetic).

For display, it is usually necessary to scale or clip pixel values to a specific numeric range, e.g. 0..255. This constraint does not hold for general image analysis operations. We might represent pixel values using floating point numbers. Computations on an image might result in negative pixel values.



2 Image Formation

2.1 Digital Image Acquisition

The sensor matrix comprises a matrix of photosensitive cells, each of which samples the image. The image sample from each cell is converted into an electrical signal. An image of the subject is then projected onto the sensor array in the camera.

The sensor array converts the image into a matrix of charges, proportional to the incoming photon flux intensity. Hence the image is sampled as pixels. This charge matrix is read out sequentially in the form of an electrical signal.

Geometric Projection from the scene to imaging plane:

- maps 3D points in the scene to 2D image points
- determines the spatial configuration of the image.

Radiometric transformations determine how light or dark the image will appear locally:

- determines pixel values according to how light or dark object surfaces are pigmented in the scene
- determines how light or dark a surface will appear dependent upon the strength and relative direction of the illumination source(s) with respect to each surface in the scene.

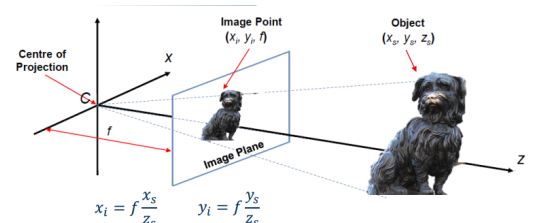
2.1.1 Bidirectional Reflectance Distribution Function(BRDF)

BRDF is material dependent and sets the relationship between surface orientation and brightness.

$$BRDF = \frac{\text{Outgoing radiance}}{\text{Incoming irradiance}}$$

For Lambert's Law (smooth) shading the BRDF is a constant $\frac{1}{\pi}$. The radiance is then proportional to $\frac{\cos(\theta)}{\pi}$, where θ is the angle between the surface normal and a point light source. For a surface of uniform material, the perceived irradiance encodes the surface direction. Not enough information to determine the surface convexity uniquely. Provides a monocular cue for depth perception. This forms the basis for *shape-from-shading* algorithms.

2.2 Pinhole Camera

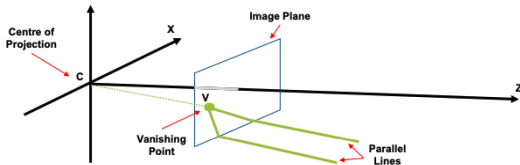


We cannot recover z uniquely from these equations. The pin-hole projection model is used extensively in computer graphics and 3D computer vision

$$\begin{bmatrix} x_i \\ y_i \end{bmatrix} = \begin{bmatrix} f & 0 \\ 0 & f \end{bmatrix} \begin{bmatrix} x_s \\ y_s \end{bmatrix} \frac{1}{z_s}$$

2.2.1 Effects of perspective

Far objects appear smaller than close ones, parallax is distant objects appearing to move more slowly.



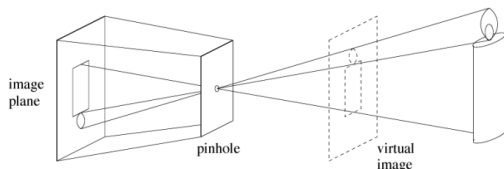
Each set of parallel lines meet at a different point, known as the vanishing point. Any two parallel lines have the same vanishing point, V . The ray from the centre of projection through V is parallel to the lines. Vanishing points are projections of points at infinity. They are a good way to spot faked images:

- Scale and perspective don't work
- Vanishing points behave badly
- Supermarket tabloids are a great source

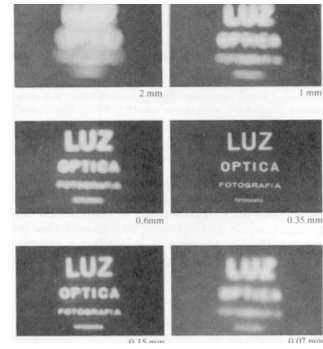
A set of parallel planes in 3D will intersect in one line, called the vanishing line. The vanishing line of the ground plane is called the horizon. Occlusion refers to light not passing through objects, therefore only the nearest point across a ray will be visible on the image plane.

The size of an object on the image plane is inversely proportional to distance, Z , to the object. Hence, varying the X or Y position of an object does not change the projected object size if Z is constant.

The locations of points in X and Y invert on the image plane (even inside our eyes the images on our retinas are upside down). Coplanar parallel lines meet on the same horizon. The projection magnification ($x/X, y/Y$) is proportional to the principal distance (i.e. the perpendicular distance from the projection centre to the imaging plane).



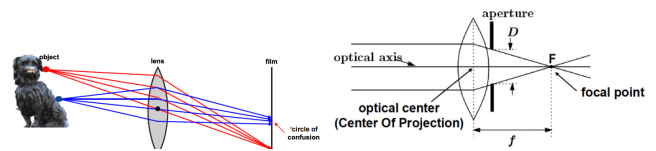
A simple, abstract camera model constrains the light travelling from a point on the object's surface to a very small angular cone, i.e. solid angle. In practice, pinhole cameras are dark, because a very small set of rays from a particular point hits the screen. If the pinhole is too big – many directions are averaged, blurring the image. If the pinhole is too small – diffraction effects blur the image.



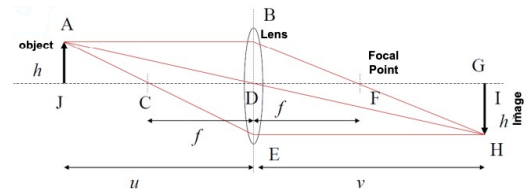
2.3 Lenses

Lenses are used to control the amount of light passed and how it maps to the image plane. Different shaped lenses will produce distortion to the image. Lenses are used to gather light, maintain a sharp image, and for their flexibility in varying the projection geometry. They allow us to obtain a high quality image more easily.

A lens focuses light onto the film. At a specific distance objects are in focus, otherwise they project to a circle of confusion on the image plane. Changing the shape of the lens changes this distance.



A lens focuses light onto a single focal point. The focal point is at a distance f beyond the plane of the lens. f is a function of the shape and index of refraction of the lens. The aperture diameter, D , restricts the incoming light.



$$\frac{AJ}{JD} = \frac{HD}{DG} \Rightarrow \frac{h}{u} = \frac{h'}{v}$$

$$\frac{BF}{BD} = \frac{HD}{DG} \Rightarrow \frac{f}{h} = \frac{v-f}{h'}$$

$$\Rightarrow \frac{f}{u} = \frac{v-f}{v} = \frac{1-f}{b} \Rightarrow \frac{1}{f} = \frac{1}{u} + \frac{1}{v}$$

Any object point satisfying this equation is in focus. Remember that f is defined as the distance from the lens centre to where parallel rays of light entering the lens converge to a focus, i.e. when $u = \infty$.

2.4 Camera Sensor Technology

Sensor semiconductor technology, i.e. CCD (Charge Coupled Device) and CMOS (Complementary Metal Oxide on Silicon).

CMOS is important as it is used for, and directly compatible with, microprocessor and memory fabrication techniques. Other processing can be integrated into CMOS imaging arrays, such as Analogue to Digital converters, or digital image filters, to construct 'smart sensors'. Both CCD and CMOS operate by converting an incoming photon into the flow of an

electron which is stored in a capacitor built into each cell of the imaging array. CMOS is now overtaking CCD in both quality and quantity.

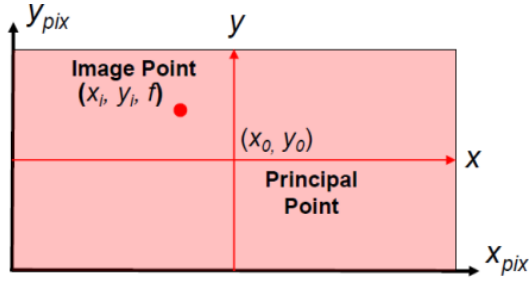
2.4.1 Colour Sensing & Distortion

Colour cameras either comprise 3 sensor array devices or single array mosaics. 3 sensor devices use a complex optical arrangement to split the incoming light into 3 channels, i.e. 3 separate but identical images, each sampled by a separate imaging array. Each channel is split into red, green, and blue component using red, green, and blue filters.

Mosaic devices place micro filters over each pixel in a regular array of red, green, blue and sometimes additional colours (e.g. IR or teal). Imperfections in lenses might cause radial distortion of an image, causing a 'barrel' or 'pin cushion' effect.

2.5 Pixel Coordinates

The origin of the pixel coordinates is typically located at a corner of the sensor array (e.g. top/bottom left). CCD/CMOS pixels may not be square (due to unequal spacing in the horizontal/vertical directions). There may also be skew factor and lens distortion (pin-cushion effect) all contributing to projection of the 3D object to different pixel coordinates.



$$\begin{aligned} x_i &= f \frac{x_s}{z_s}, & x_{pix} &= k_x x_i + x_0 = k_x f \frac{x_s}{z_s} + x_0 \\ y_i &= f \frac{y_s}{z_s}, & y_{pix} &= k_y y_i + y_0 = k_y f \frac{y_s}{z_s} + y_0 \end{aligned}$$

$$\begin{aligned} \alpha_x &= f k_x, & x_{pix} &= \frac{\alpha_x x_s + z_s x_0}{z_s} \\ \alpha_y &= f k_y, & y_{pix} &= \frac{\alpha_y y_s + z_s y_0}{z_s} \end{aligned}$$

2.5.1 Homogeneous Coordinates

Homogeneous coordinates are typically used for projective geometry. The advantage is that coordinates of points, including those at infinity, can be represented using finite coordinates. For point (x, y) in the Euclidean plane, its representation in the projective plane is simply $(x, y, 1)$ and for homogeneous coordinates, we have $(u, v, w) = (a u, a v, a w)$ for $a \neq 0$, and its equivalent in the Euclidean plane is $(u/w, v/w)$.

The use of homogeneous coordinates can simplify a lot of the projective representations:

$$\begin{aligned} x_i &= f \frac{x_s}{z_s} \\ y_i &= f \frac{y_s}{z_s} \end{aligned} \quad \begin{bmatrix} u \\ v \\ w \end{bmatrix} = \begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x_s \\ y_s \\ z_s \\ 1 \end{bmatrix} \quad \begin{aligned} x_i &= \frac{u}{w} \\ y_i &= \frac{v}{w} \end{aligned}$$

Euclidean coordinates $\Rightarrow$ Homogeneous coordinates $\Rightarrow$ Back to Euclidean coordinates.

2.5.2 Internal Coordinates – Calibration Matrix

Given the following general camera projection:

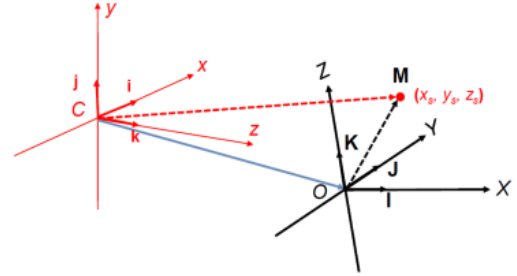
$$\begin{bmatrix} u \\ v \\ w \end{bmatrix} = \begin{bmatrix} \alpha_x & s & x_0 & 0 \\ 0 & \alpha_y & y_0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x_s \\ y_s \\ z_s \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} \alpha_x & s & x_0 & 0 \\ 0 & \alpha_y & y_0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} \alpha_x & s & x_0 \\ 0 & \alpha_y & y_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} = K[I | 0]$$

K is called the calibration matrix (3x3 upper triangular matrix), representing the internal parameters of the camera and has five DoF.

2.6 Camera Transformation Matrix

The use of homogeneous coordinates gives a convenient way of representing points in two coordinate systems. For computer vision, it is necessary to consider projection of points to a camera with arbitrary position and orientation (pose) in 3D space. We use unit vectors, I, J, K to represent the world coordinate system, and unit vectors i, j, k to represent the camera coordinate system.



We often use rotation matrices to represent coordinate transforms; this in fact can be more conveniently represented as the inner product of coordinate unit vectors, for example, when you rotate along the Z axis, we have:

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} I \cdot \vec{x} + J \cdot \vec{y} + K \cdot \vec{z} \\ J \cdot \vec{x} + J \cdot \vec{y} + J \cdot \vec{z} \\ K \cdot \vec{x} + K \cdot \vec{y} + K \cdot \vec{z} \end{bmatrix} = \begin{bmatrix} z \cos \alpha - y \sin \alpha + 0 \\ z \sin \alpha + y \cos \alpha + 0 \\ 0 + 0 + 1 \end{bmatrix}$$

2.6.1 Camera Transformation

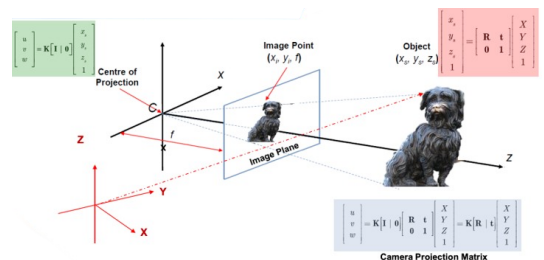
$$\begin{bmatrix} x_i \\ y_i \\ z_i \end{bmatrix} = \begin{bmatrix} 1 \cdot X + 1 \cdot Y + 1 \cdot Z \\ 1 \cdot X + 1 \cdot Y + 1 \cdot Z \\ 1 \cdot X + 1 \cdot Y + 1 \cdot Z \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

Rotation matrix R (its align global to local camera coordinates)

Homogeneous Coordinates:

$$\begin{bmatrix} x_i \\ y_i \\ z_i \end{bmatrix} = R \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} + t \quad \begin{bmatrix} x_i \\ y_i \\ z_i \end{bmatrix} = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

In practice...



The camera projection matrix defines how a point in the world is projected to the image plane in homogeneous coordinates. P has 11 degrees of freedom (5 from calibration matrix K , 3 from R , and 3 from t).

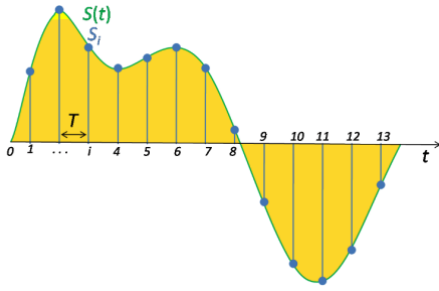
P is a 3×4 matrix with relatively general elements and its inverse can be represented as $P^+ = P^T(PP^T)^{-1}$ (the pseudo-inverse, as it is not square).

If x is the projection of point X to the camera coordinates, then $x = PX$ defines the ray that links C with X . Any point on that ray $P + x + \lambda C$ will project to the same point on the image plane.

3 Frequency Domain

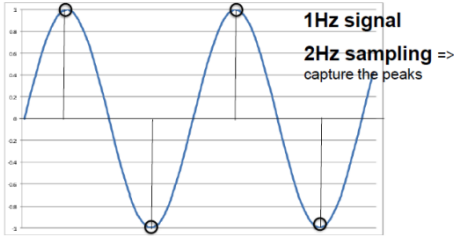
3.1 Sampling and Quantisation

For digital processing, analog signals produced by the cameras and sensors need to be quantised. Quantisation takes place in both amplitude and space/time. Any signal can be perfectly reconstructed as soon as the sampling rate is high enough.



3.1.1 Nyquist Frequency

The sampling rate has to be at least twice the highest frequency of the original signal: $f_s \geq 2f_0$. f_s is the sampling frequency and f_0 is the frequency of the original signal.

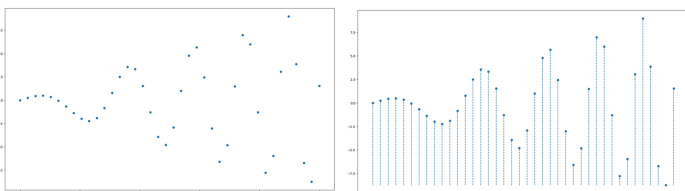


3.1.2 (Anti-) Aliasing

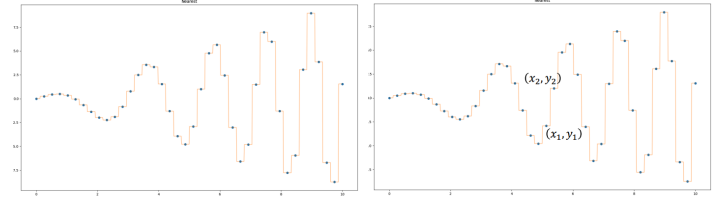
Aliasing is the result of not having samples at the right rate. High-frequency components 'fold over'. In video, it will appear like fast moving objects have stopped moving (wagon wheel effect).

Aliasing can be removed with filtering. Filtering removes high-frequency components, ensuring that there is no frequency component higher than the Nyquist frequency. There is a trade-off between anti-aliasing and sharpness.

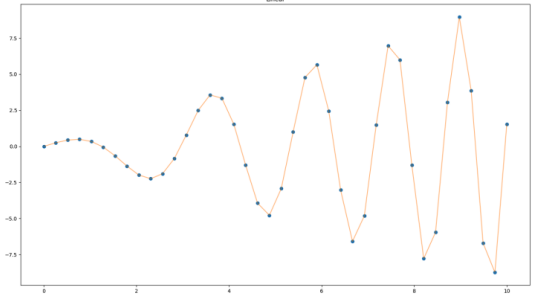
3.2 Interpolation & Resampling Operations



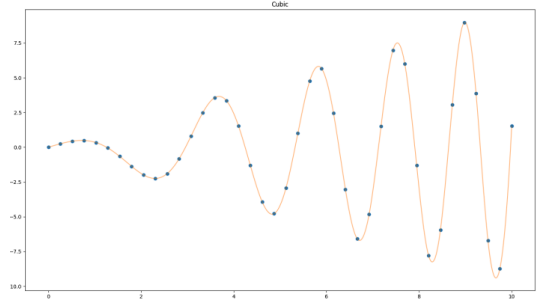
3.2.1 Nearest Neighbour Interpolation



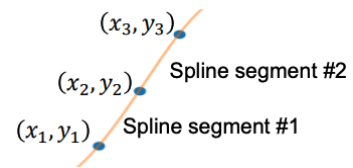
3.2.2 Linear Interpolation



3.2.3 Cubic Spline Interpolation



The curve is passed through all specified data points. The first derivative is continuous at interior points, as is the second derivative. Boundary conditions are specified at free ends.



$$\begin{aligned} \text{Spline Segment \#1} \\ y &= a_1(x - x_1)^3 + b_1(x - x_1)^2 + c_1(x - x_1) + d_1 \\ y' &= 3a_1(x - x_1)^2 + 2b_1(x - x_1) + c_1 \\ y'' &= 6a_1(x - x_1) + 2b_1 \end{aligned}$$

$$\begin{aligned} \text{Spline Segment \#2} \\ y &= a_2(x - x_2)^3 + b_2(x - x_2)^2 + c_2(x - x_2) + d_2 \\ y' &= 3a_2(x - x_2)^2 + 2b_2(x - x_2) + c_2 \\ y'' &= 6a_2(x - x_2) + 2b_2 \end{aligned}$$

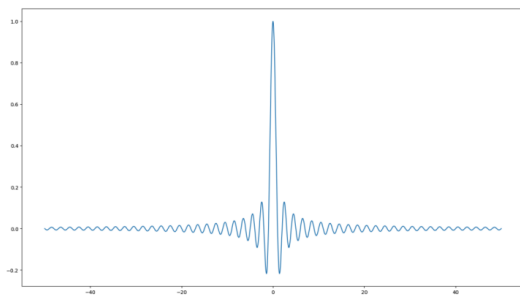
Constraints for spline segment #1: $y'' = y_1''$ at $x = x_1$, $y'' = y_2''$ at $x = x_2$. y_2' in spline 1 must equal y_2' in spline 2.

3.3 Signal Reconstruction

We can reconstruct a function by summing up 'impulse response' functions placed on the sample points, weighted by

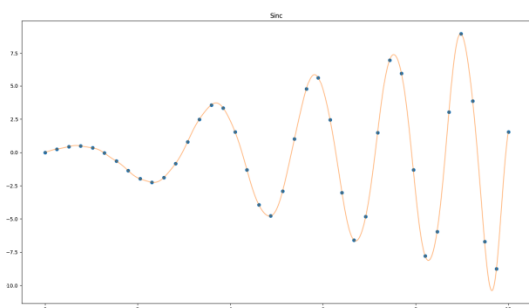
the sample values.

$$f(x) = \sum_{k=-\infty}^{\infty} f[k] h(t - kT)$$



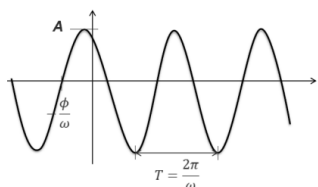
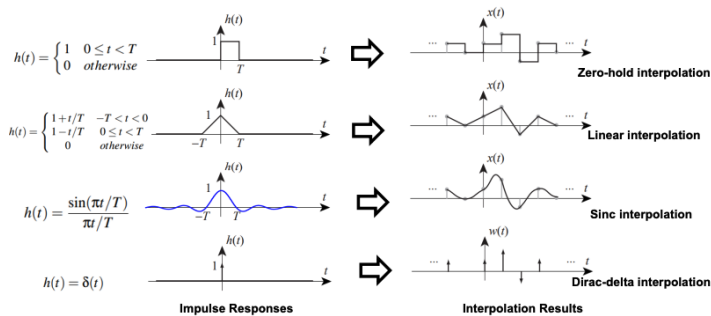
The reconstruction of the original function f from a discrete signal F is ideal when the impulse response is the sinc function

$$\text{sinc}(t) = \frac{\sin(\pi t)}{\pi t}$$



3.3.1 Impulse Responses

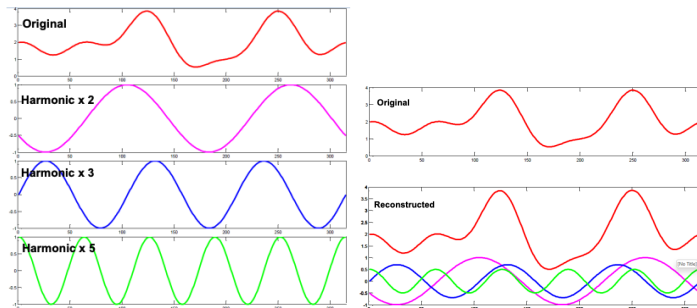
$$f(x) = \sum_{k=-\infty}^{\infty} f[k] h(t - kT)$$



$$f(t) = A \sin(\omega t + \varphi) = A \sin\left(\omega \left(t + \frac{\varphi}{\omega}\right)\right)$$

3.3.2 Signal Reconstruction – Harmonics

Using harmonic analysis it is possible to reconstruct arbitrary signals by superimposing combinations of sines and cosines of different A , ω , φ .

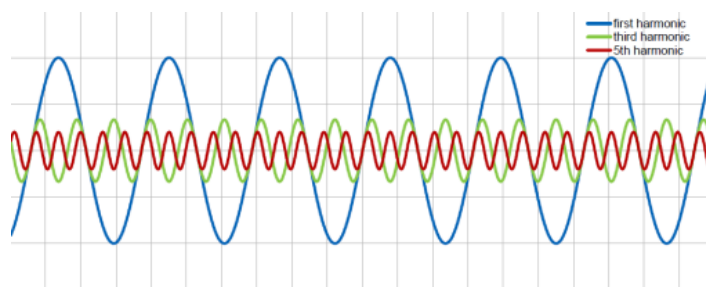


3.4 Fourier Series/ Fourier Synthesis

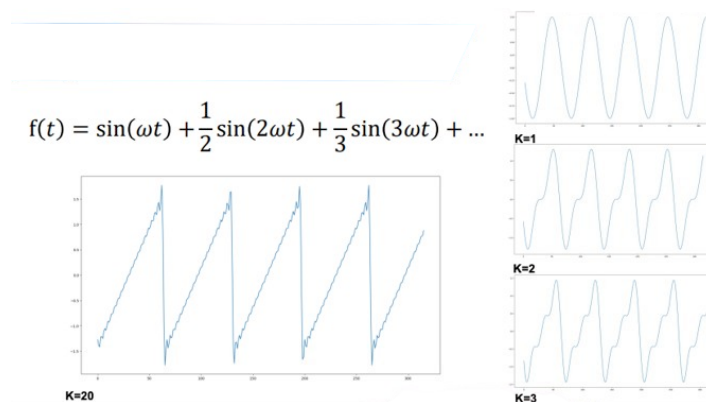
Almost any periodic function can be expressed as a linear combination of sine/cosine wave form.

$$f(t) = \sin(\omega t) + \frac{1}{3} \sin(3\omega t) + \frac{1}{5} \sin(5\omega t) + \dots$$

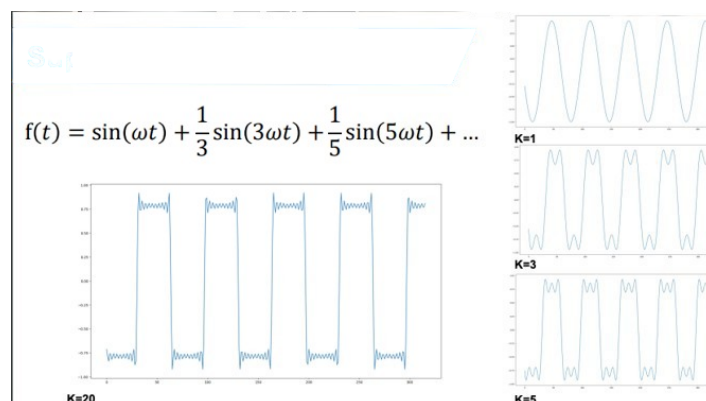
where ω , 3ω , 5ω are the first, third and fifth harmonics.



3.4.1 Super-position of Sine Waves



Here we see the result of adding sine waves that are integer multiples of a fundamental frequency. Adding each harmonic creates increasingly complex waveforms, and at 20 - each with decreasing amplitude - the result approximates a sawtooth wave



Similarly, here we are using only **odd** harmonics. Each additional harmonic begins to flatten the top and bottom of the waveforms. Somewhat at 5, and especially at 20, the result approximates a square wave. Note that we can see rippling near the transitions, this is Gibbs phenomenon.

3.4.2 Continuous Fourier Transform

It expresses a function $F(x)$ as an infinite sum of simple waves (sinusoids) with distinct frequencies, amplitudes and phases.

$$\hat{F}(f) = \int_{-\infty}^{\infty} F(x) e^{-i 2\pi f x} dx$$

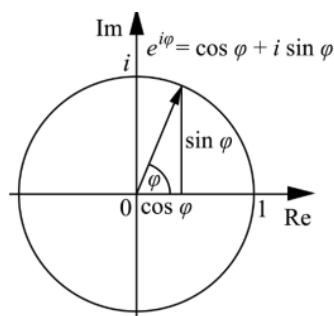
frequency

Euler's formula for complex exponentials

$$e^{i\theta} = \cos(\theta) + i \sin(\theta)$$

Inversion of Fourier Transform:

$$F(x) = \int_{-\infty}^{\infty} \hat{F}(f) e^{i 2\pi f x} df$$

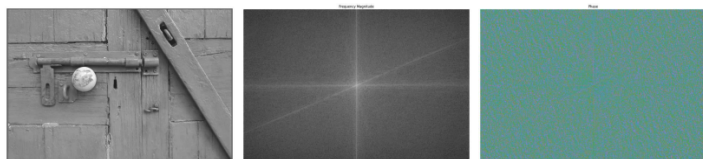


3.4.3 Discrete Fourier Transform

The Fourier transform is defined for periodic, continuous functions. For a discrete vector of data: $X = x_0, \dots, x_{N-1}$:

$$X_k = \sum_{j=0}^{N-1} x_j e^{-i 2\pi k \frac{j}{N}}$$

A two dimensional transform can be separated into a series of one dimensional transforms.



3.5 Fourier Transform and Convolution

Spatial Domain (x)	↔	Frequency Domain (u)
$g = f * h$	↔	$G = FH$
$g = fh$	↔	$G = F * H$

So, we can find $g(x)$ by Fourier transform.

g	=	f	*	h
↑		↓		↓
IFT		FT		FT
↓		↑		↑
G	=	F	×	H

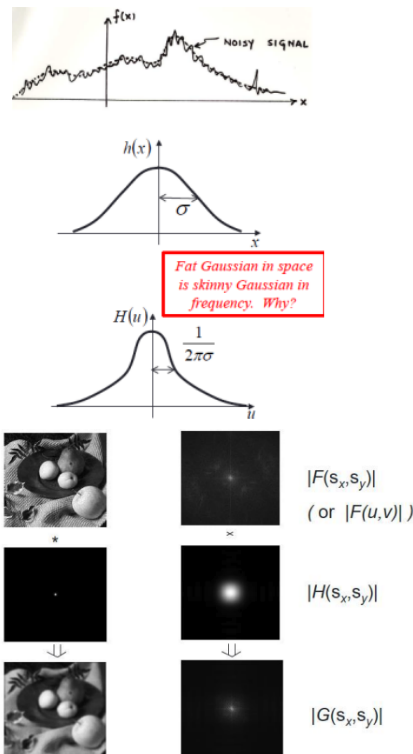
Let's say we want a smoothed function of $f(x)$, where $g(x) = f(x) * h(x)$. Let's use a Gaussian kernel

$$h(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left[-\frac{1}{2} \frac{x^2}{\sigma^2}\right]$$

The Fourier transform of a Gaussian is a Gaussian

$$H(u) = \exp\left[-\frac{1}{2} (2\pi u)^2 \sigma^2\right]$$

Convolution in space is multiplication in frequency: $G(u) = F(u) H(u)$. $H(u)$ attenuates high frequencies in $F(u)$ (low pass filter).



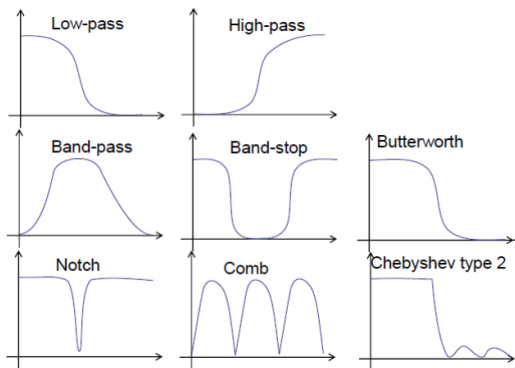
Applications of Fourier Transform:

- Image editing – removing textures, moire patterns, and similar distortions.
- Image filtering – images can be smoothed (low pass filtered) or detail enhanced (high pass filtering) by simple transforms of the magnitude spectrum.
- Image compression – split images into chunks, compute DFT (or the very similar DCT – discrete cosine transform) and discard high frequencies. This is how JPEG compression works, with some refinements in coding the missing frequencies efficiently.
- Pattern recognition – Certain patterns, such as periodic patterns of texture, are easier to distinguish in the frequency (spatial) domain. Image patches can be transformed via the DFT and stored/looked up in a database via similarity metrics.
- Image alignment – images can be precisely aligned using the phase difference between two images (e.g. two adjacent frames in a video sequence).

4 Filtering

There are multiple types of filter, for example,

- Low-pass filter
- High-pass filter
- Band pass filter
- Band stop filter
- Notch filter
- Comb filter
- Butterworth filter
- Chebyshev filter



4.1 Point and Area Filters

Point filters are filters that modify pixels independently, without information from surrounding pixels. Point filters include operations like gamma correction, which applies an independent scaling to each pixel.

Area filters modify pixels taking into account a limited neighbourhood of nearby pixels. Area filters include operations like smoothing and edge detection.

There are two basic divisions of area filters:

- Linear filters, where each pixel $I[x, y]$ is a weighted sum of the neighbouring pixel values. Example:

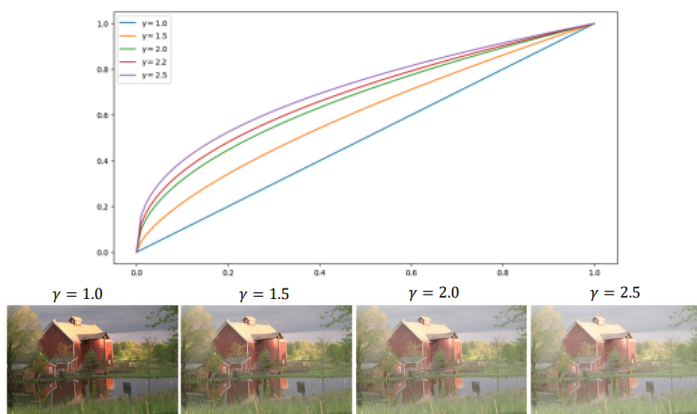
$$I[x, y] = \alpha_0 I[x, y] + \alpha_1 I[x - 1, y] + \alpha_2 I[x + 1, y]$$

- Non-linear filters, where each pixel $I[x, y]$ is defined by any other function of neighbouring pixels. Example:

$$I[x, y] = \min(I[x, y], I[x, y + 1])$$

4.1.1 Point Filters

Point filters apply a function $f(x)$ to each pixel in an image independently. Gamma correction: non-linear scaling of brightness: $I[x, y] = [x, y]^{\frac{1}{\gamma}}$



Quantisation - Reducing the number of levels used in an image



Colour Transformations (Y: luminosity/brightness, i: in-phase, Q: quadrature. From RGB to YIQ

$$\begin{bmatrix} Y \\ I \\ Q \end{bmatrix} \approx \begin{bmatrix} 0.299 & 0.587 & 0.133 \\ 0.5959 & -0.2746 & -0.3213 \\ 0.2115 & -0.5227 & 0.3112 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix} \Rightarrow \text{Transformation}$$

From YIQ to RGB

$$\text{Transformation} \Rightarrow \begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 1 & 0.956 & 0.619 \\ 1 & -0.272 & -0.647 \\ 1 & -1.106 & 1.703 \end{bmatrix} \begin{bmatrix} Y \\ I \\ Q \end{bmatrix}$$

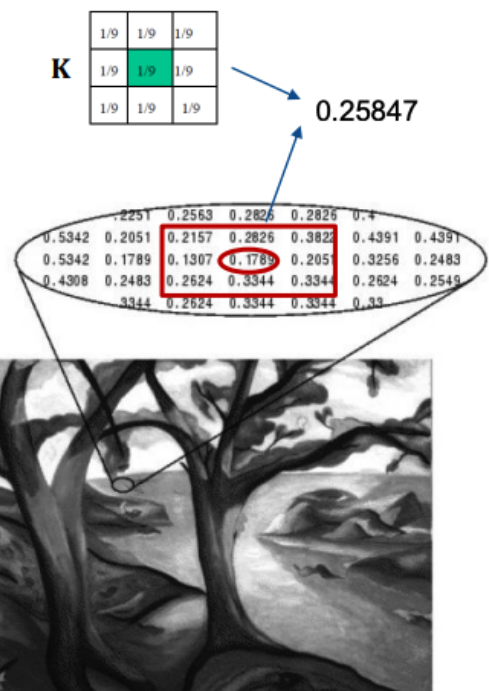


4.1.2 Area Filters - Linear Filters

The output is a weighted sum of the neighbouring values.

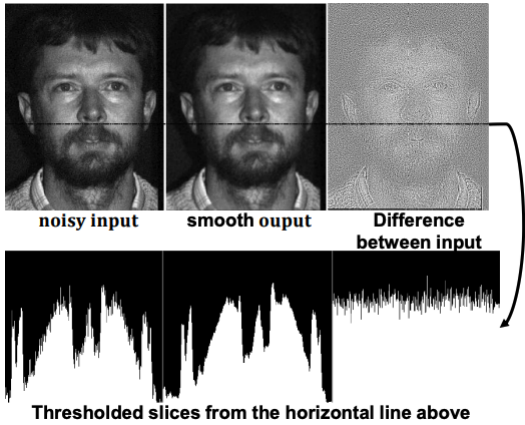
$$f(I[x, y]) = \sum_i \sum_j I[x + i, y + j] K_{ij}$$

Where we sum over i rows and j columns around $[x, y]$. $K_{i,j}$ are the coefficients matrix and it is called Kernel. The kernel fully specifies the linear filter.



Effects of Local Smoothing The difference image contains both noise and input image characteristics. If the filter had only smoothed out noise, the difference image should only contain the noise that had been removed.

In general, filtering operations applied to an image to not conserve the information contained in an image. Filters have to be optimised to be "best" for a specific task.



4.2 Convolution

4.2.1 1D Convolution

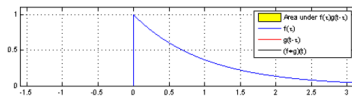
Convolution is a mathematical operation, Continuous domain:

$$y(t) = (f * g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t - \tau)d\tau$$

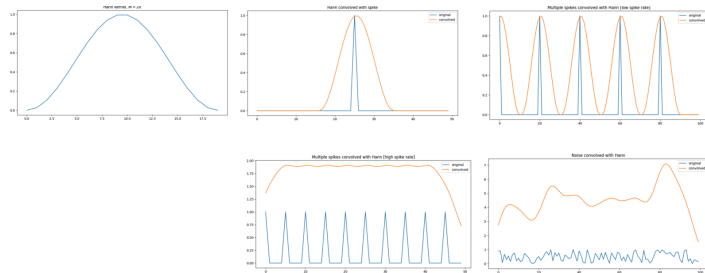
Discrete domain:

$$y(t) = (f * g)(t) = \sum f(\tau)g(t - \tau)$$

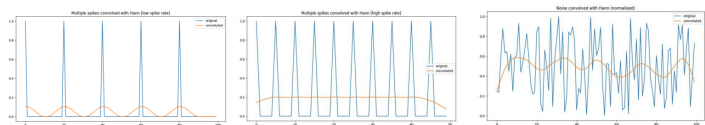
Where f is the input function and g is the kernel or filter



Examples The following are examples of 1D convolutions



Kernel Normalisation Kernel normalisation is useful in order not to change the intensity of the signal, i.e. the brightness of an image $K_N = \frac{K}{\sum K}$



Algebraic Properties The convolution operator commutes and associates $f * g = g * f$
 $f * (g * h) = (f * g) * h$
Distributes over addition : $f * (g + h) = f * g + f * h$.
Allows for efficient implementation of filtering

4.2.2 2D Convolution

$$H(x,y) = \sum_{j=1}^{height} \sum_{i=1}^{width} I(i,j)M(x-i,y-j)$$

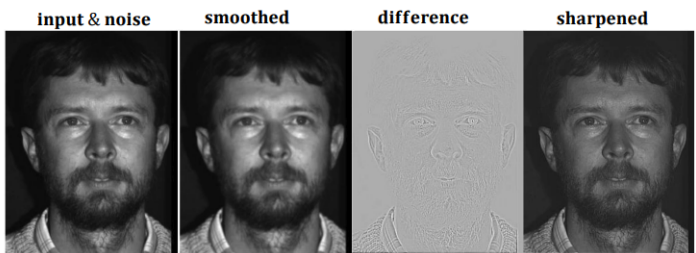
A kernel convolution is the basic operation applied to an image to filter it. This process involves taking a matrix of numbers, multiplying these by the image data below in the kernel window. The summed result is the output value of the pixel centred on the kernel. The process is repeated for every image location, till a new output image has been created.

Issues with convolution

- Time Complexity: Convolution is $O(MN)$ for signals of length M convolved with signals of length N .
- Changed Size: The result of a convolution of an image of size $W \times H$ with a kernel of size $W_k \times H_k$ is $W \times H \times W_k \times H_k$
- Edge Effects: Convolution is well-defined for infinite signals. In practice, we have finite data; we need to define what happens at the edges of the signal. The appropriate choice depends on the signal properties. Common approaches are to assume:
 - Zero: Everything is zero or another fixed constant beyond the edges
 - Flat: The value at the edge is repeated forever
 - Wraparound: The signal wraps around at the edge to the other edge (only appropriate for periodic signals)
 - Mirror: The signal is reflected at the edge, and we read values in reverse order.

4.3 Unsharp Masking

When an image is smoothed and subtracted from the original the steep edges, i.e. high frequencies are left over. These image components contribute to making an image appear sharp. If we compute the edge residuals and add these back to the original image it has the effect of making the original image look sharper.



Unsharp mask operation:

- Edge residuals = input - smoothed.
- Unsharp masking = Edge residuals + input = 2*input - Smoothed

We can express the unsharp mask operation in a single kernel

$$9 * \begin{pmatrix} 0 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 0 \end{pmatrix} - \begin{pmatrix} 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \end{pmatrix} = \begin{pmatrix} -1 & -1 & -1 \\ -1 & 17 & -1 \\ -1 & -1 & -1 \end{pmatrix}$$

Input x 2 **Smoothing** **Combined & Scaled Kernel**

4.4 Edge Detection

Edges, or discontinuities in intensity are often associated with physical discontinuities in the world, for example caused by:

- changes in the surface slope of the subject
- occlusion of one surface behind another
- changes in material properties that result in a change in albedo(reflectivity)

Accordingly edges have perceptual significance, i.e. they relate to something important in the real world for the observer

Edges as Image Gradients Gradients can be estimated as the partial derivatives of the image intensity

$$\frac{\partial I(x, y)}{\partial x}, \frac{\partial I(x, y)}{\partial y}$$

For continuous functions:

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x-h)}{2h},$$

For discrete domains: $(f[x] - f[x-1])\Delta H$

This can be approximated as:

$$\frac{(f[x] - f[x-1]) + (f[x+1] - f[x])}{2}$$

4.4.1 Roberts Kernel

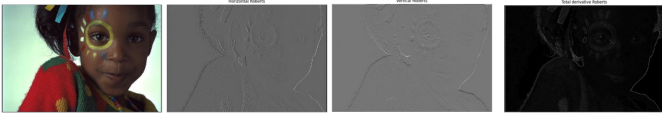
The Roberts kernel is a basic finite difference kernel. It consists of:

- A horizontal kernel $K_h = [1 \ -1]$
- A vertical kernel $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$

The total derivative at any given pixel is the combination of the horizontal and vertical kernels.

$$dI = \sqrt{(K_h * I)^2 + (K_v * I)^2}$$

Other functions can also apply to combine the two kernels, i.e. average, mean square, or sum of absolute values.

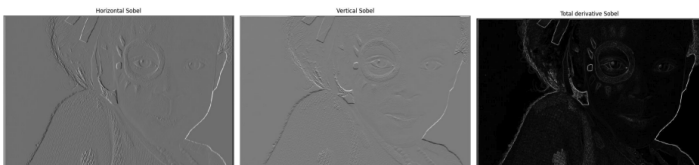


4.4.2 Sobel Kernel

The Sobel kernel

$$\begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}_v \quad \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}_h^T$$

It combines edge detection with smoothing and it is symmetrical to the central pixel. It handles noise better than Roberts kernel



4.4.3 Oriented Kernels

The Roberts and Sobel kernels are separable, i.e. they can be applied sequentially along the image rows as two separate 1D filter operations. To obtain a single edge map, the two edge images must be combined pixel-wise. Kernels can also be tuned in orientation, i.e. enhance edges in each of the 8 compass directions

$$\begin{bmatrix} -1 & 1 \\ -1 & 1 \end{bmatrix} \quad \begin{bmatrix} -1 & -1 \\ 1 & 1 \end{bmatrix} \quad \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$$

4.4.4 Problems with Edge Filters

Resolution sensitivity: The filters sensitivity to edges depends on the resolution of the images. You can't easily rescale the Roberts kernel for a new image size.

Noise: The edge detecting filters are a kind of high pass filter, a filter that removes slowly varying components and keeps the high frequency ones. This enhances edges, but also enhances noise.

False edges: The edge detecting filters cannot distinguish true, perceptual edges from lightness variations that are irrelevant. Post processing is needed to parse the edge enhanced images into meaningful data, and throw away glitches like very short, isolated edge fragments, or coalesce broken edge elements into a single continuous edge.

5 Filtering II

2D Convolutions are expensive, recall that an image of size $W \times H$ convolved with a kernel of size $W^* \times H^*$ takes $O(WH W^* H^*)$ time to compute. We can do two 1D convolutions instead (one on the rows and one on the columns, or vice versa) as convolution commutes. This is much faster for larger kernels, as the time complexity is $O(WH W^*) + O(WH H^*)$.

A kernel is *separable* if it can be written as an outer product of two vectors $K = k_a k_b^T$. This can be tested by checking the rank of a matrix, i.e. finding the number of linearly independent rows; separable filters have rank 1.

5.1 Convolution theorem

The convolution theorem states that the Fourier transform of the convolution is equal to the (element-wise) product of the Fourier transform of two signals, i.e. we can compute the convolution by taking products of the frequency/spatial frequency domain, then transforming back to the time or spatial domain.

Time Complexity if Fast Fourier Transform (FFT) is used:

$$\approx O(N_x \log N_x + N_x \log N_y),$$

where $N_x = WH$ and $N_y = W^* H^*$.

The convolution theorem tells us how the spatial-domain convolution affects the frequency domain. We can analyse the effects of filters before applying them. For example, when downsizing an image by ratio r , we must ensure there are no frequency components $> \frac{f_n}{r}$ before dropping pixels. Otherwise, we will alias

The pre-filtering kernel K must have $DFT(K)$ so that $|DFT(K)| = 0, > \frac{f_n}{r}$.

Linear Filters in the Frequency Domain A linear filter can change the amplitude of frequency components, but it can *never introduce new frequencies*. If a frequency is not present in a signal, no linear filter can make it appear. Only non-linear filters can introduce new frequencies.

Truncated Pseudo-Gaussian Filters True Gaussian blurring is not practical on a large digital image, because every single pixel contributes to the blurred value of every other pixel. Truncated versions are commonly used instead (e.g. Hamming, Hann, Blackman-Harris, etc.).

5.2 System Identification

We can filter images to get all kinds of effects. What if we want to find out how an unknown black-box system is affecting our imaging? For example, how a new lens in a camera is blurring the light out?

If we assume that the system is linear, then its transformation will be a convolution. Many systems can be well approximated with a linear model, and the maths is particularly easy for linear systems. So our problem reduces to estimating K , a convolution kernel, from data observed from an imaging system.

5.3 The Dirac Impulse

To understand what convolution is doing, we use the Dirac impulse function. Named after Paul Dirac, δ is defined as an infinitely tall and thin pulse whose area sums to unity.

It is defined to exist at the origin in time $\delta(t)$ at $t=0$, $\delta(x)$ at $x = 0$ in 1D, or $\delta(x, y)$ at $x, y = 0, 0$ in 2D. The mathematical definition of the Dirac impulse is then:

$$1 = \int_{-\infty}^{\infty} \delta(x) dx \quad \text{or in 2D:} \quad 1 = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} \delta(x, y) dx dy.$$

Conceptually, δ is the mathematical equivalent of a hammer; striking a system with it causes the system to output its characteristic response.

In a linear electronic network, if we apply a voltage spike to the filter input, the filter will output a good approximation to its characteristic response, i.e. the convolution function.

Since the convolution function determines how the filter will shape input signals we can use this knowledge to predict how the filter will respond to any signal input.

The Dirac impulse is an important analytic tool, both a mathematical probe and a probe that can be applied to real systems if we can apply a short pulse to the input.

The convolution process is defined in terms of the Dirac impulse to a filter kernel M : $M * \delta$ where $*$ is the convolution operator. The discrete form of the convolution is as we saw:

$$H(x, y) = \sum_{j=1}^{\text{height}} \sum_{i=1}^{\text{width}} I(i, j) M(x - i, y - j),$$

where H is the output image, I is the input image, and M is the convolution kernel. According to the above property of the Dirac impulse, it should be possible to convolve an image containing an impulse. The resulting output image should just contain a copy of the convolution kernel used for the definitions to hold true

So, what does an image of a Dirac impulse look like? Since the value of the Dirac impulse has a unit area and digital images comprise intensity values integrated over a finite area, a not unreasonable interpretation might be a **unit valued pixel**

5.3.1 Dirac Impulse Test for Kernel Identification

1. Generate an image of odd dimensions greater than twice the expected kernel dimensions (to avoid edge effects).

2. This image should contain all zero values except for a single pixel (at the centre), containing the value 1.0.
3. Convolve the system to be tested on this image.
4. Observe the non-zero values in the output; this pattern is the unknown kernel.

In optical systems, the kernel is called the **point spread function** (PSF) as the convolution will spread out a point of light as above.

5.4 Use of PSFs

Convolution is in practice a reversible operation. However, if we consider the convolution theorem, then,

$$FT(f * g) = FT(f) FT(g),$$

so one might imagine writing

$$FT(f) = \frac{FT(f * g)}{FT(g)},$$

i.e. we could recover an unconvolved function f by dividing the FT of the convolved function $f * t$ by the FT of g and inverting the Fourier transform. In practice, $FT(g)$ may have lots of zeros (making the division undefined), and this division is incredibly sensitive to any noise in measuring $f * g$ or g .

5.4.1 Convolution Properties

Kernels can also be tuned in size.

- Small kernels suffice to pass sharp edges containing high spatial frequencies
- Large kernels (with many rows and columns) are required to pass more blurred edges containing low spatial frequencies.

Linear kernels can be combined directly into a single kernel. Multiple convolutions can be combined:

$$I * M_1 * M_2 = I * (M_1 * M_2).$$

Kernels should sum to unity to avoid offset/gain in the filtered image. Note that image noise tends to occupy the high spatial frequencies, so gradient filters can amplify image noise which can be problematic.

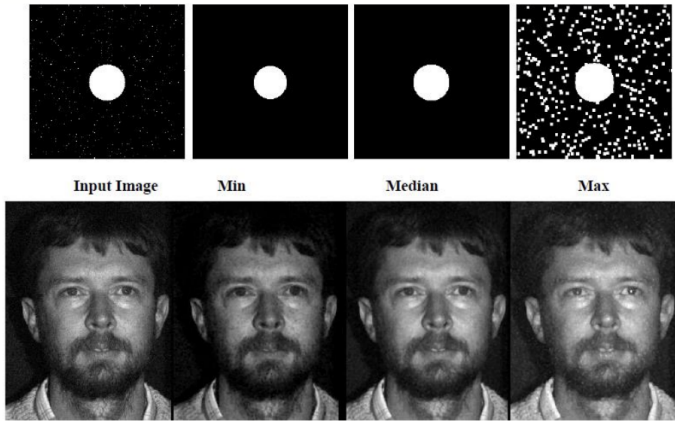
5.4.2 Implementation Issues

- The usual implementation approach is to compute the convolution directly for each pixel in the input image. This is a problem at image boundaries however...
- A good solution is to copy the input image into a slightly larger image to accommodate the kernel "falling outside the input image"
- E.g. for a 3x3 kernel, add two extra rows and columns and set these to the values of the border pixels of the input image.
- Compute the convolution only for the area of the original image. No loop tests are now required to check if the kernel is going to cross the image boundary.
- Alternatively, kernels can be computed by multiplying the input image by each kernel coefficient and storing in a separate image. These images are then shifted and added - this is very efficient, especially for symmetric kernels, but is costly in memory

5.5 Non-Linear Filters

Rank order filters rank the pixel values in magnitude order under a local window and then select a given rank as the output pixel. Most common forms are **min**, **max**, and **median**, though any rank can be selected within the range of the kernel.

They are robust, they work well for both binary and grey-level images. Median filtering is useful for removing impulsive noise without excessive blurring; min and max filters can shrink or expand binary features respectively, and select dark/light grey level image features. Below are some examples of rank filtering

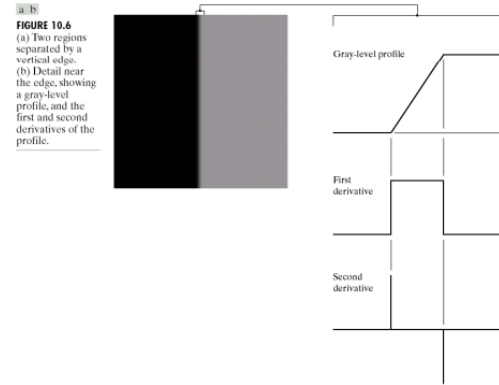


- Possibly produced by real object boundaries, surface markings, or illumination/surface direction changes,
- Potentially manifested by changes in second-order image properties (e.g. boundaries between textures).

The interaction of illumination with changes in surface directions can generate edges.

Consider edges produced by simple grey-level discontinuities. We have seen operators that compute edge strength, we want to produce an explicit (symbolic) edge representation.

Remember gradient descent? Keep this in mind



5.5.1 Rank Order Filtering

- An Output rank $>$ median $\rightarrow$ *opening operation* (Increases size of the disc, favours brighter pixels).
- Output rank $<$ median $\rightarrow$ *closing operation* (Decreases size of the disc, favours darker pixels).
- Output rank = median $\rightarrow$ preserves region sizes and removes isolated compact features, favours uniform image regions.

Opening and closing operations can be combined to suppress or enhance image features according to their relative brightness.

For byte images with a small kernel (say, less than 16x16 window) or floating point images;

- use a sorted linked list of window values as it slides.
- Extract the old window values and insert new values to the list as the window traverses the image.
- This gives a direct index to each rank in the list.

For large window Byte images, use a look-up table based on the window histogram

Other non-linear filters:

- *Bilateral filtering*: Blur pixels *less* if there is large disparity in the window to preserve edges while smoothing noisy areas.
- *Morphological operations*: e.g. erosion/dilation (common in binary images). Erosion shrinks the 'on' portion, Dilation expands it. For non-binary images, erosion is replaced by min and dilation is replaced by max operations.

6 Edge Detection

Edges describe essential characteristics of a structure. An edge is:

- A perceptually significant contour,
- Generated by a change (discontinuity) in the image,

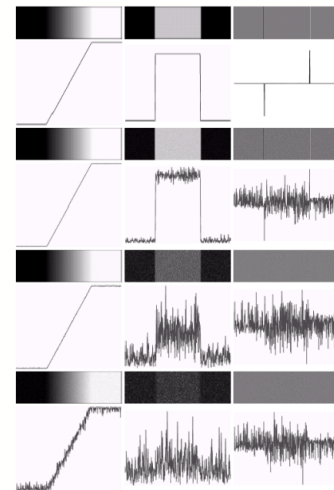
6.1 Effect on Noise on Gradient

Column 1: images and grey-level profiles of a ramp edge corrupted by random noise (rows 2,3,4).

Column 2: first-derivative images and (grey level) profiles.

Column 3: second-derivative images and profiles.

Image noise in rows 2-4 has an average amplitude of 0.1, 1.0, and 10.0 grey levels respectively



The outputs from the mask pairs are combined into a single gradient image. Combine mask outputs by either computing the average of the modulus of each mask or the root of the sum of the squares of each output. We usually combine mask outputs (e.g. horizontal and vertical) into a single gradient image, for example by summing magnitudes or taking the root-sum-of-squares:

$$|G_x| \text{ and } |G_y| \rightarrow \sqrt{G_x^2 + G_y^2} \text{ or } |G_x| + |G_y|.$$



Original image top left, vertical image gradient $|Gy|$ top right, horizontal image gradient $|Gx|$ bottom left, and combined gradients $|Gx| + |Gy|$ bottom right.

6.2 Gaussian Smoothing

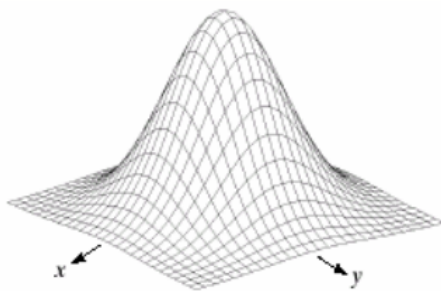
Roberts and Sobel operators estimate the gradient but they also emphasise noise in the image. A simple uniform averaging filter can suppress noise.

In theory this can be combined with the gradient kernel to produce less noisy edges, but the simple averaging also blurs the image and resulting edges excessively. Averaging using centre weighting is a compromise that helps to reduce blurring while still suppressing image noise.

A Gaussian **function** provides a better centre-weighted average. A 3D plot of the Gaussian kernel is below.

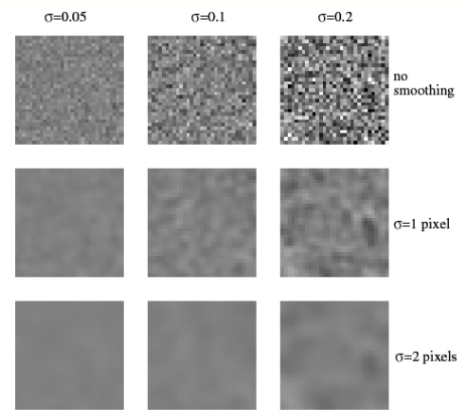
- The mesh node heights correspond to the kernel values
- The area under the Gaussian function is normalised to unity
- Recall that unity is required to avoid scaling the image
- The width is controlled by a single parameter, σ
- Think of the Gaussian Kernel as a centre weighted average

It **will** blur, but less than simple uniform averaging



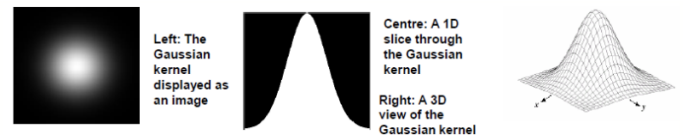
Generally, you can expect pixels to "be like" neighbours, surfaces turn slowly, and there are few abrupt reflectance changes). Noise processes can be expected to be independent from pixel to pixel. This implies smoothing suppresses noise, for appropriate noise models.

Scaling the σ parameter in the Gaussian, as it goes up, the more pixels are involved in the average, so the image gets more blurred and the noise is more suppressed. Below are various widths of Gaussian smoothing applied to Gaussian noise images.



6.3 The Gaussian Filter

The Gaussian filter has a bell-shaped kernel and is essentially a centre-weighted averaging low pass filter.



The equation of the Gaussian in x,y centered at 0,0 is:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} = \left[\frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{x^2}{2\sigma^2}} \right] \times \left[\frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{y^2}{2\sigma^2}} \right].$$

Hence $G(x,y)$ is separable, meaning it can be computed as two 1D convolutions

6.4 Combining Smoothing and Differentiation

Key issues: Noise

- We should smooth *before* differentiating.
- We can use a *derivative of Gaussian* filter; by associativity of convolution, we convolve once rather than smoothing then differentiating separately.

6.4.1 LoGs and DoGs

The second spatial derivative can also be used for edge detection. The sum of the second partial derivatives of an image is called the Laplacian. The Gaussian smoothed second derivative kernel is called the **Laplacian of a Gaussian (LoG)**. A quick approximation is the **Difference of Gaussians (DoG)**.

6.5 Canny Edge Detector

John Canny proposed an optimal edge detection scheme that would give the best combination of

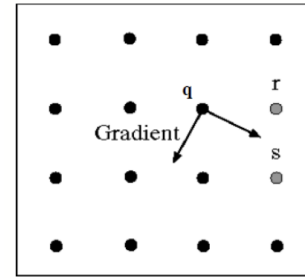
- *Error rate*, responding only to edges and should find them all, none should be missed.
- *Good localisation*, the distance between edge pixels found by the edge detector and actual edge should be as small as possible
- *Minimal response*, the edge detector should not identify multiple edge pixels here only a single edge exists

It turns out that the first derivative of the Gaussian provides a good approximation, example of gradients in x and y shown below

$$\frac{\partial G(x, y)}{\partial x} = \left(\frac{-x}{\sigma^2} \right) \frac{1}{2\pi\sigma^2} e^{-\left(\frac{x^2+y^2}{2\sigma^2} \right)} \quad \frac{\partial G(x, y)}{\partial y} = \left(\frac{-y}{\sigma^2} \right) \frac{1}{2\pi\sigma^2} e^{-\left(\frac{x^2+y^2}{2\sigma^2} \right)}$$

This has 3 major issues

- The gradient magnitude at different scales is different, which should we choose? (the scale of the smoothing filter affects derivative estimates and also the semantics of the edges recovered)
- The gradient magnitude is large along thick trail, how do we identify the significant points?
- How do we link the relevant points up the curve?



6.6 Gradient Based Edge Detection

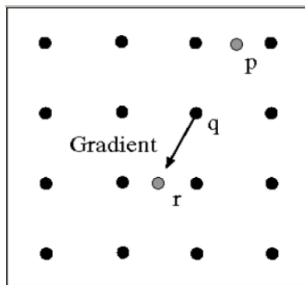
Form an estimate of the image gradient in x and y using horizontal and vertical Gaussian derivative filter. From this, get the gradient magnitude and direction. identify image points where the value of the gradient magnitude is maximal and in the direction perpendicular to the edge and is large. These are edge points.

The gradient magnitude tends to be large along thick trails in an image, typically, We want to condense these trails into curves of representative edge points. We wish to park points along the curve where magnitude is biggest. We can look for a maximum along a slice normal to the curve (non-maximum suppression), these points form a curve, then, we are left with two algorithmic issues:

At which point is the maximum, and where is the next one?

6.7 Non-Maximum Suppression

At q , we have a maximum if the value is larger than those at both p and r . **Interpolate to get these values.** Non-maximum suppression obtains points where the gradient magnitude is at a maximum along the direction of the gradient. The figure below shows how we reconstruct the gradient magnitude. The dots are the pixel grid.



We are at pixel q , attempting to determine whether the gradient is at a maximum, the gradient direction through q does not pass through any convenient pixels in the forward or backward direction. We must interpolate to obtain the values of the gradient magnitude at p and r , if the value at q is large than both, q is an edge,

Typically the magnitude values are reconstructed with a linear interpolator, which in this case would use the pixels to the left and right of p and r respectively to interpolate values at those points. If q has a higher intensity than its two directional neighbours, its an edge point.

Predicting the Next Edge Point Assume the marked point is an edge point, we construct the tangent to the edge curve, which is normal to the gradient at that point, use this to predict the next point, either r or s .

Above, we sketch how to find candidates for the next edge point, given q is an edge point. An appropriate search direction is perpendicular to the gradient, so that points s and r should be considered for the next edge point. Notice that in principle, we don't need to restrict ourselves to pixel points on the image grid, we know where the predicted position lies between s and r , we could again interpolate to obtain gradient values for points off the grid.

Edge Following: Pseudo-code

```
while there are unvisited points with high gradient:
  find point that is local max in gradient direction
    while possible:
      1. predict next location from the
         edge tangent
      2. check if that location is a local max
      3. check if gradient at the max is
         large enough
      4. record the point, mark neighbors visited,
         update current point
    end
```

6.8 Remaining Issues

We need to check that the maximum value of the gradient is sufficiently large. *Hysteresis thresholding* addresses potential drop-outs:

- Use a high threshold T_h to start edge curves and a low threshold T_l to continue them.
- Any gradient pixel higher than T_h is presumed to be an edge pixel and is marked as such
- Search for gradient pixels connected to the pixels $> T_h$ that are also $> T_l$ and label these as well

Notice that something is happening at corners; Scale affects contrast; Edges aren't bounding contours.

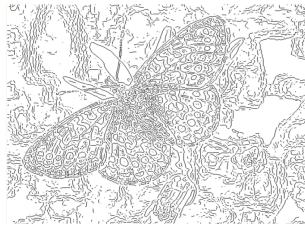
6.9 Hysteresis Thresholding

Hysteresis refers to a process that depends on the history of a state, not just its current value. In the context of edge processing, it refers to keeping edge values only if they make sense in context. The standard way to do this is a morphological operation, which looks at the standard 3x3 neighbourhood, and follows these rules:

- Strong edges remain strong.
- None stays none
- Weak edges becomes strong **if at least 1 neighbour is strong**, otherwise becomes none

The result is a binary image with only strong and none values, all weak edges have been classified according to their spatial context

Below are examples of applying different scales/thresholds:



(Top-left) Original Image, (Top-right) Fine scale, High threshold, (Bottom-left) Coarse scale, High threshold, (Bottom-right) Coarse scale, Low threshold.

6.10 Corner Detection

Most edge detection functions struggle to detect corners (the intersections of edges) reliably, separate algorithms are used. "Corners" is often interpreted quite liberally to mean key points.

Moravec's algorithm detects corners by defining them to be pixel regions with low self-similarity, i.e., parts of the image which are very different from their neighbours.

The Harris operator and Shi-Tomasi operator improve the reliability of corner detection over Moravec's algorithm, they do not suffer from the directional bias that the Moravec algorithm suffers from, they are **isotropic**

7 Image Segmentation

In order to analyse an image in terms of content:

- Break the image into isolated components, i.e. segment the image
- Segments can be processed into symbols or tokens
- Subsequent reasoning can then be applied to the symbolic form.

Assume that image components comprise

- Regions of uniform intrinsic properties, grey level or colour being the simplest;
- Could be uniform texture, which is harder to analyse;
- Edges where uniform regions meet,

So in some sense edges and regions are dual (redundant) representations

7.1 Segmentation

Image segmentation is not grounded in a general theory. Appropriate segmentation can be achieved by ad hoc means (heuristics) for specific tasks.

Simple underlying assumptions that clearly do **NOT** hold generally, but can serve as the basis for useful segmentation algorithms:

- Uniform image regions correspond to uniform surfaces in the scene
- Edges correspond to the interface at real object boundaries.

So uniform regions and their edges may have both physical and also perceptual significance

Segmentation is the initial step for many tasks, for example, you can segment an image to determine its: **Position**, by computing its centre location; **Size**, by counting the number of pixels forming its area; **Shape**, by computation applied to

its boundary.

Many vision-based tasks require this segmentation step:

- Object detection and recognition: inspection systems, Content-Based Image Retrieval (CBIR)
- Post-production manipulation of media images: isolating actors and props (matte pulling) for special effects, insertion of different backgrounds
- Document analysis, text recognition- character isolation
- Medical image interpretation and measurement - isolation of tissue types in MRI scan slices, x-ray analysis
- and more...

A number of different situations will be considered here, namely:

- Simple two region segmentation — A foreground object set against a background
- K region segmentation, with minimum region size (area) specified
- Sampled region segmentation — e.g. a user can point to the desired regions which are then segmented automatically
- A reference image is available, i.e. a background image, containing the scene but without object(s) to be isolated
- Colour segmentation
- Adaptive segmentation to allow segmentation parameters to vary over the image, as opposed to using global settings.

Note that the image to be segmented need not have been produced by a white light camera, e.g. MRI images or X-ray images

7.2 Histogram Construction

Analysing the grey level properties of an image is the pixel value histogram, i.e. grey level histogram or simply image histogram. Computed by counting the number of each shade of grey found in the image and displaying this as a classical histogram.

If we have N shades of grey in an image, we can store the histogram in a 1D array of N elements:

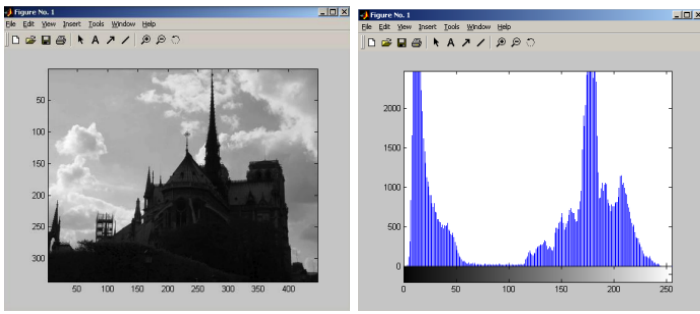
- for $t = 1..N$; $H(t) = 0$; initialise H to contain zeros
- for each pixel in image $I(X,y)$: $H(g) = H(g) + 1$; increment the count for the current shade of grey

The array H now contains the number of instances of each shade of grey found in the image.

7.3 Histogram Properties

Basic approach: consider an image region to have uniform grey-level properties. Near uniform grey-levels (real-world situation) is reflected in a peak in the image histogram centred at the mean grey-level value. There will be a cluster of grey levels varying about this mean.

Each uniform region will be represented as a separate peak if it is a sufficiently large (in area) with respect to other peaks, and is of a sufficiently different mean value to avoid merging with other peaks



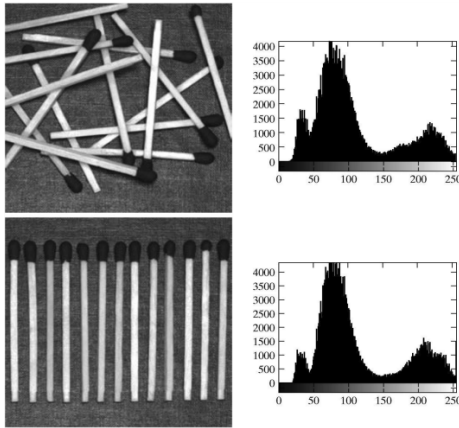
The histogram of the left example is shown on the right. Select output pixels O less than intensity threshold, t :

- $O(x,y) = I(x,y) < t$; to select dark region
- $O(x,y) = I(x,y) > t$; to select light region

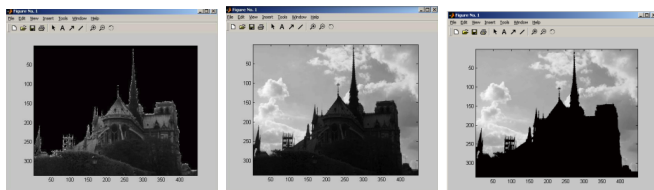
The relationship between the image and its histogram:

- Darker regions appear as peaks on the the histogram left
- Lighter regions appear as peaks on the histogram right
- Histogram valleys often correspond to the edge pixels between regions

All explicit spatial information is lost in the histogram - try to relate regions in the image to peaks in the histogram. The histogram only preserves the relative frequency of grey levels.



Using thresholds, we can extract the dark or light regions of images, for example



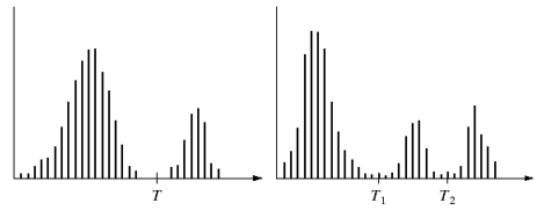
centre - original image, left - dark region, right - light region

7.3.1 Histogram Modality

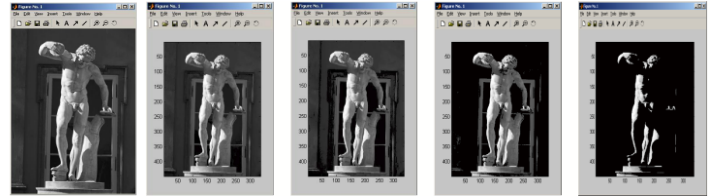
Each peak in the image corresponds to an almost uniform region, the left histogram indicates two image regions present, while the right histogram indicates three image regions present.

The highest point in each peak centres on the mean shade of grey of the region.

How spread each histogram peak is indicates how widely the shades of grey in the corresponding image region vary about the region's mean shade of grey.



This has some limitations, for example, the following image has no single threshold which captures the entire figure. Setting a double threshold can help ($O(x,y) = t_2 > I(x,y) > t_1$) to select an intermediate region, but perhaps not in this case.



input, with outputs thresholded at 25, 45, 75, 160 ($L \rightarrow R$)

7.4 The Histogram Function

The modality of a histogram refers to the number of peaks present. The number of peaks is assumed to indicate the number of significant regions in the image.

A **unimodal** histogram contains a single peak indicating that the image comprises a single quasi-uniform region.

A **bimodal** histogram contains a two peaks indicating that the image comprises two quasi-uniform regions. This is the easiest situation to deal with and can serve many situations. Text and diagram segmentation from scanned documents are a good example.

A **multimodal** histogram contains an arbitrary number of peaks indicating that the image is complex in structure, the greater the number of peaks, the less likely that the image will comprise simple quasi-uniform regions.

The *broader* each peak, the greater to range of shades in the corresponding image and it can be harder to segment. The *narrower* the peak, the more homogenous the corresponding image region and the easier it is to segment.

7.4.1 Histogram as a Probability Distribution

If we divide through the histogram (element-by-element) values by the total number of pixels in the image we now have the probability of selecting a pixel of a given shade at random from the image:

- $p(g) = h(g)/(N * M)$ = probability of grey level g
- $h(g)$ is the histogram and N & M are the image dimensions in pixels

Normalising in this manner produces a histogram, with values in the range 0..1. This is called the grey-level Probability Density Function (PDF). The PDF indicates the density (likelihood) of finding a pixel of a given shade of grey when inspecting the image locally.

This concept extends to colour images in several ways:

- We can compute separate 1D histograms for each of the red, green, and blue colour planes.
- We can convert to a different colour space and histogram this.

- We can compute a single 3D histogram for red, green, and blue simultaneously to get a 3D histogram providing a PDF of each unique colour.

We can investigate how we can use the histogram to segment an image

7.4.2 First Approach: Bimodal Histogram

It is helpful to know something about the structure of the images we can expect to segment:

- The potential contrast phase (light on dark or vice versa) of the target region to be isolated.
- The nominal size of the target region, expressed either as a percentage of the whole image or a fraction.
- This fraction is the prior probability of selecting at random a pixel from the whole image that is contained within the target region

Knowing the above we can simply search from the target shade end of the histogram, summing the fractions of the image contained in each (normalised) histogram bin until this corresponds most closely to the fraction of the image we believe the target to occupy.

We then set out segmentation threshold to segment at the histogram grey level that separates out the desired fraction of the image. This approach is known as the p-tile method, the p-tile approach is simple but not very robust.

Bimodal Histogram Search Naive histogram search (single threshold):

- Search for the histogram peaks, i.e. bin values bounded by lower values
- $(H(n) > H(n+1)) \ \& \ (H(n) > H(n-1))$
- Rank these by peak value order
- Between the highest pair of peaks search for the lowest histogram valley, i.e. bin values bounded by lower values $(H(n) < H(n+1)) \ \& \ (H(n) < H(n-1))$
- The lowest valley between the highest pair of peaks is the threshold

Problem: histogram may be noisy with many peaks close together, i.e. the second peak may be right next to the first. Synthetic images may have several maximum bins of the same height.

Solution: Filter, e.g. smooth the input image - results in a smaller range of histogram values and may have to smooth quite severely - computationally expensive and distorts the image. Or we can filter the histogram - fewer local valleys, but may reduce the peaks too much.

7.5 Estimating a Threshold

An alternative approach to segmentation can be based on the following heuristic:

1. Select an initial estimate for t
2. Segment the image using t , this will produce two groups of pixels
 - G_1 consisting of all pixels $> t$
 - G_2 consisting of all pixels $\leq t$
3. Compute the average grey level values, μ_1 and μ_2 for the pixels in regions G_1 and G_2
4. Compute a new threshold value: $t = (\mu_1 + \mu_2)/2$
5. Repeat steps 2 through 4 until the difference is t in successive iterations is less than a predefined parameter t_o

7.5.1 Generalising the Estimator to K Regions

Suppose we know that we have 3 regions, we need to find 2 thresholds (t_1 and t_2) to separate these 3 regions

1. Either select initial values for the 2 thresholds or initial values for the mean grey levels of each region (μ_1 , μ_2 and μ_3)
2. As before segment the image into 3 regions using the 2 current thresholds
3. compute new mean values (μ_1 , μ_2 and μ_3) for each region
4. compute new thresholds:
5. $t_1 = (\mu_1 + \mu_2)/2$, $t_2 = (\mu_2 + \mu_3)/2$
6. Halt if the thresholds (t_1 and t_2) have stabilised else iterate from 1

Straightforward to extend this algorithm to K regions and this is known as K means clustering

7.5.2 User Defined Regions

In certain situations it may be known in advance where specific regions in an image are likely to be located, e.g. in an inspection system parts are likely to be presented to the inspection camera in the same position, therefore definite foreground and background regions can be determined in advance.

In photo booths the centre of the image is likely to contain the subjects face and the top corners contain the background.

In a photo editing program a user might be given the option to select the centre location of object and the background regions.

Knowing where samples of the pixels of the regions of interest can be obtained means that good initial estimates for the mean values for these regions can be obtained and the previous algorithm then applied (or even a simple one-shot version based on an initial pass of the algorithm without iterating)

7.6 Background Subtraction

If a *reference* is available, this can be used to provide a better segmentation. A reference image comprises a view of the scene that does not contain the desired (target) object. When the target object enters the scene, a target image is captured.

The reference image can be numerically subtracted (pixel-by-pixel) from the target image to reveal differences corresponding to changes in the captured images. These changes are usually predominantly caused by the presence of the target object, if:

- The camera has not moved since the reference image was captured
- Everything else in the field of view has not moved/changed
- No additional non-target objects have entered the field of view
- The ambient illumination has not changed
- The presence of the target has not caused significant changes to the scene illumination by secondary reflections from the target object surge onto non-target object surfaces.

The difference image can then be thresholded as before (using the techniques presented) to find the region corresponding to the target object. Note that negative intensity values may be generated in the difference image

- so take the modulus of the difference, or rescale to a positive range of grey levels range

- Where the target and the background have the same grey level, holes will appear in the target
- Ambient illumination changes are difficult to control

It is often necessary to devise methods for updating the reference image to accommodate changes in the scene and ambient illumination. The usual approach is to store a running average of images: $T_n = (1 - \alpha)T_{n-1} + \alpha I_n$ where α [0..1] sets the fraction of the input image I that is accumulated into the average reference image T for the n th input sample (initialise T with a known good reference image)



1. Background 2. Target 3. subtracted image 4. thresholded $t=10$ 5. median 7×7

7.7 Colour Segmentation

The most direct colour segmentation process assumes either a background image is available, or that the background colour has been sampled and the mean background colour μ_{bg} has been computed from this sample

Compare the vector distance, $D(x, y)$ of the RGB values of each pixel from the input image to the mean background using

$$D(x, y) = \sqrt{(r(x, y) - \mu_r)^2 + (g(x, y) - \mu_g)^2 + (b(x, y) - \mu_b)^2}$$

or to each pixel of a background reference image using

$$\sqrt{(r(x, y) - r_{bg}(x, y))^2 + (g(x, y) - g_{bg}(x, y))^2 + (b(x, y) - b_{bg}(x, y))^2}$$

In both cases a distance image $D(x, y)$ has now been formed that can be segmented using grey-level techniques.

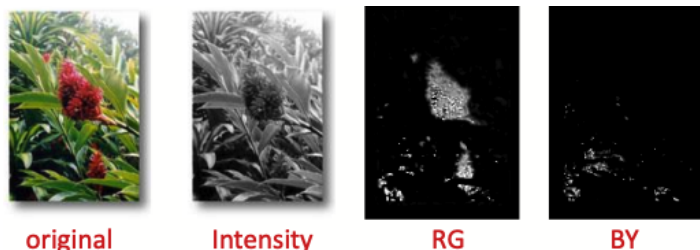
It is sometimes advantageous to convert from RGB colour space to Hue, Saturation and Intensity and make the comparison based on Hue alone. This affords a degree of invariance to illumination changes, by basic the segmentation only on the primary colour information

7.7.1 Colour Space Conversion

To increase the contrast between foreground and background objects, we can change to a different colour space. below transformations:

- Intensity = (red + green + blue) / 3
- RG = (red - green) / (red + green)
- BY = (blue - yellow) / (blue + yellow)

note: yellow = red + green.



Red target shows strongly against green background in RG colour space, some remaining bright spots due to division by zero artefacts

7.7.2 Illumination Effects

Illumination gradients have the effects of smearing the histogram peaks. In fact the background could become lighter than the foreground in some areas, making global segmentation based on a single threshold impossible. The solution is to break the image into sub-images and then threshold these independently.

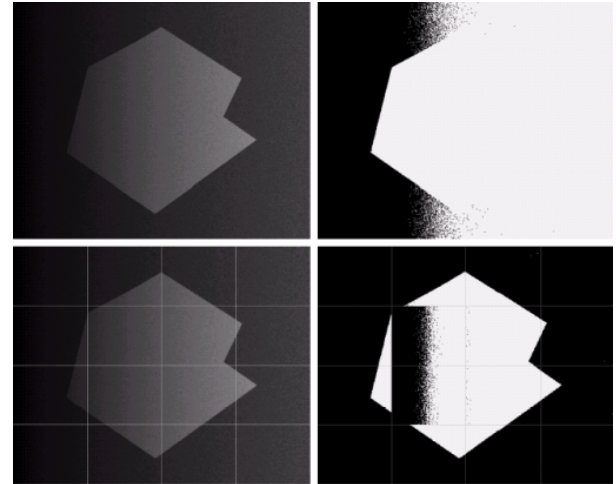
7.7.3 Basic Adaptive Subdivision

Subdivide the image into 4 sub-images and sub-divide again (16 sub-images). Compute the variance (spread or range of values) in each sub-image.

If the variance is greater than or equal to v ($v = 100$ here), compute a threshold on this block as it likely contains two regions.

If $v < 100$ then the block is probably almost uniform, treat all such blocks together as a composite image and compute a threshold as before.

The above test represents a homogeneity criterion



top-left: original image. top-right: result of global thresholding.
bot-left: Image subdivided into individual subimages. bot-right: Result of adaptive thresholding

This shows the resulting improvement, two segments failed to segment as they contained too small an area of background to be detected. The histogram looked essentially unimodal for these blocks.

Increasing the subdivision improves the spatial resolution of the segmentation.

8 Multiresolution and Image Pyramid Approaches

Image scale consider your ability to recognise objects at differing distances and cope with seeing them at different apparent sizes. Consider grouping similar objects together such that at some level of resolution they are indistinguishable.

For example in the far distance you might just distinguish a face, as you get closer, you determine it to be male/female, closer still you might recognise who it is, and even closer, you might notice a slight bruise.

Scale is important when attempting to detect, identify, and classify images of objects. Scale can also be important when

tracking objects regardless of their size or how close they become to the observer.

Multi-Resolution Important from a computational perspective, the idea revolves around processing an image within a "pyramid" data structure.

This involves taking the image and shrinking it by a fixed amount, say x2, to create a smaller image then recursively repeating this process to generate a pyramid of increasingly smaller images. Therefore, each smaller image contains a progressively coarser version of the original image.

We can apply scale independent processing to this multiresolution image data structure to achieve efficient computation over a range of image scales. Such techniques are also useful for guiding efficient image search processes and scale independent feature extraction used in object recognition and database search.

8.1 Image Scale Issues

As we have seen, scale is also an important consideration when extracting image features such as edges. In general, we don't know the range of scales of structures that will be present in an image in advance. We know that:

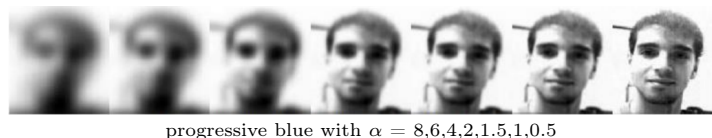
- The smallest feature is limited by the size of a pixel (inner structure)
- The largest feature is limited by the size of the whole image (outer structure)
- A continuum of possible scales can exist in an image between the above extremes

Clearly, we can't construct filters for every possible scale, therefore we need a method to allow us to apply the same algorithm (or kernel) regardless of the scale of image feature present.

Now let's examine the basis for a unified framework for managing image analysis over differing scales.

8.1.1 Progressing Scale Change

Varying the blurring parameter α for a Gaussian filter allows us to compute filters with varying degrees of attenuation of high frequencies (low-pass). Applying this set of filters to an image gives a set of images containing features according to their spatial scale.



Think of the Gaussian kernel as a centre weighted averaging filter. Examining the above sequence of filtered images from coarse-to-fine scale we see that as high frequencies are introduced (with reducing scale), new levels of complexity appear in the image.

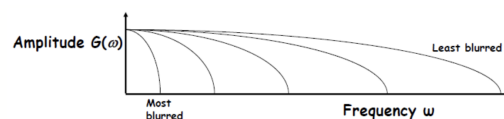
However, each new feature that appears in a given scale then persists and evolves into subsequent scales, i.e. each new scale contains all the information of the previous scale. Hence it is possible to track or trace these features from coarse-to-fine images.

The evolution of image features with scale is referred to as the image deep structure and understanding this is the subject of cutting edge research.

Low-Pass Blur Frequency response of low-pass Gaussian filters: High frequencies are attenuated according to

$$G(\omega) = e^{\frac{-\omega^2 \sigma^2}{2}}$$

Where $\omega = 2\pi f$ and f = cycles/pixels



8.2 Space Scale

8.2.1 Decomposition

It is more useful to attempt to isolate the new information to the scale at which it appears, i.e. represent the information difference between scales. To do this an image can be deconstructed into a series of bandpasses.

Each bandpass comprises only a range of image spatial frequencies: c.f. an audio system that breaks an audio signal into bass, mid, and treble, or an audio equaliser with even more channels.

Information is now partially separated between spatial scales, there is overlap between scales, this gives us continuity when comparing scales



8.2.2 Properties

With each new scale the only information passed is associated with the bandpass frequencies of that scale, and this increases in resolution/complexity as scale size decreases.

Again, each new feature that appears in a given scale then persists into subsequent scales. This is the case because image information tends to be contained in edges and these edges contain spatial frequency harmonics present over a range of scales.

However, a feature will have the strongest response, i.e. contrast, in the scale that is most closely matched to size (i.e. scale) of the feature - for example, look at the eyes in the image below. This is an identical concept to tuning a radio to receive specific frequencies, only we are now working with 2D signals and 2D spatial frequencies.



It is possible to track or trace features across scales - with less interaction between scale. For example, when finding edges we could employ a coarse-to-fine strategy first to find large strong edges and then associate (i.e. trace across scales) these with their finer versions at higher resolution scales

8.2.3 Building Scale Space

We could construct the above set of images by subtracting images filtered with Gaussian kernel of differing σ . Alternatively, we can form a kernel comprising the difference of two Gaussian functions of varying differing σ and then apply this combined kernel to the input image.

Varying each of the two σ parameters for the Difference of Gaussians (DOG) filter allows us to compute filters with different bandpasses (sub-bands in some terminology) that can be considered to be operating at different spatial scales. Clearly, the closer the two Gaussians are in size the narrower the range of frequencies passed by the DOG filter and vice versa

It turns out that a **good ratio of σ to choose for the Gaussians is 1.6**, in this case the difference of Gaussians kernel closely approximates a Laplacian of Gaussians (LOG) kernel

8.2.4 Effect of Filter Scale (size)

The gradient magnitude at different scales (controlled by σ) is different. Different features are emphasised according to scale.

- At fine scales fine detail is preserved at the expense of emphasising noise as well.
- At coarse scales noise is suppressed, but fine details are lost while strong features remain.

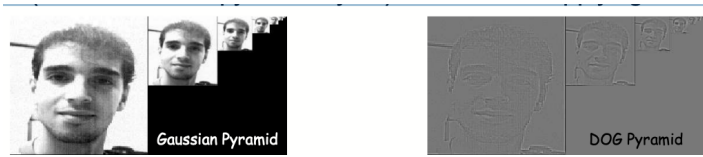
Which scale should we choose? This subject is dealt with using image pyramids

8.3 Image Pyramids

Having decomposed an image into a low passed (or band passed) image set, it is possible to represent this set within a multi-resolution pyramid. Pyramids provide increased storage efficiency & a uniform basis for analysis over scales.

Below are examples of Gaussian (low-pass) and DOG or LOG (bandpass) image pyramids. Expanding and summing the pyramid levels in the DOG pyramid reconstructs the original image.

It is possible to represent each image scale with a reduced number of pixel samples according to the Nyquist sampling limit. The coarser the scale, the lower the maximum (significant) frequency present, therefore the fewer samples (pixels) required in total to capture two samples/cycle as per the Nyquist sampling criterion.



Each level in the pyramid can be treated equivalently for analysis to achieve scale independence: e.g. find edges (in DOG or LOG pyramid layers) at each level applying same algorithm to all levels

Tools to Build Pyramids Gaussian Pyramid basic operation:

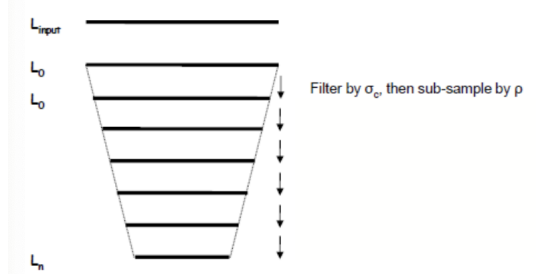
- Gaussian filter $\rightarrow$ sub-sample by factor of $\rho \rightarrow$ repeat for next level as before
- A sub-sampling (decimation) factor, $\rho=2$ is most often used
- $p = \sqrt{2}$ is also a particularly useful reduction factor for image matching.

Note, a sub-sampling factor of ρ reduces the effective size of σ by ρ . It is possible to combine sub-sampling and convolution into a single operation, e.g. if sub-sampling by a factor of 2, simply step the kernel over every second pixel of the input image to output a half resolution image.

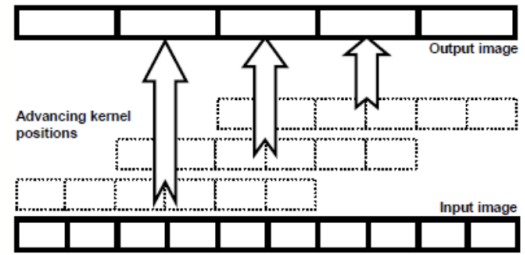
8.3.1 Pyramid Construction

Construction of a Regular Pyramid:

- Apply an initial filter of σ_{init}
- Filter subsequent levels by the same σ_c
- Sub-sample by the same ρ factor at each level



For combined filtering and sub-sampling, sub-sample twice by stepping filter kernel over every second pixel, then filter subsequent levels by the same σ_c



8.3.2 Gaussian Pyramid Requirements

We would like to have the same apparent level of blur at every level in the pyramid – recall that in a digital imaging system, a Dirac impulse stimulus comprises a unit (1.0 valued) pixel set against a black (0.0 valued) background.

Applying any linear kernel filter to a Dirac stimulus will generate an image of the kernel itself.

So when we apply the pyramid to a Dirac impulse image, the impulse response at each level in the pyramid should be the same, i.e.

$$\sigma_0 = \sigma_1 = \sigma_2 \dots \sigma_i$$

Since the response is the same at each level, we can apply the same analysis algorithms at each level such that they respond in the same manner.

All that is changing is the scale (size) of the structures from the original image. We need to work out for a given reduction ρ what is the relationship to σ_c and σ_{init}

Each level of the pyramid illustrated previously can be implemented efficiently by repeated convolution of a Gaussian filter σ_c , sub-sampled by a factor ρ

By inspection, we can observe that for the i^{th} pyramid level we can find the blur, σ_i , for the current level as follows

$$\sigma_i = \frac{1}{p} \sqrt{\sigma_{i-1}^2 - \sigma_c^2}$$

Substituting $\sigma_i = \sigma_0$ and $\sigma_{i-1} = \sigma_0$ in the above equation gives: $\sigma_c = \sigma_0 \sqrt{p^2 - 1}$, where σ_0 should ideally equal the intrinsic blur (pre-existing) in the input image for σ_c to remain constant at each level. This avoids the necessity for an initial stage of Gaussian convolution σ_{init} .

For an octave pyramid $\rho=2$ and σ increases in power of 2 for each level, therefore $\sigma_c = \sigma_0 \sqrt{3}$ in the case of a pyramid with $\rho = \sqrt{2}$, $\sigma_c = \sigma_0$.

The effective σ had they not been reduced is $\sigma_{effective} = \sigma_0 \rho^i$, note: i is numbered such that 0 corresponds to the finest scale

8.3.3 Blur in Each Gaussian Pyramid Level

No intrinsic blur has been assumed in the previous pyramid construction figure. However, any image captured using a real (optical) imaging device will typically exhibit a finite point spread function that may well approximate a Gaussian sufficiently for the purpose of pyramid construction.

Assuming that we have measured the intrinsic input image blur, σ_{image} , the correct value of σ_0 is simply:

$$\sigma_0 = \sqrt{\sigma_{init}^2 - \sigma_{image}^2}$$

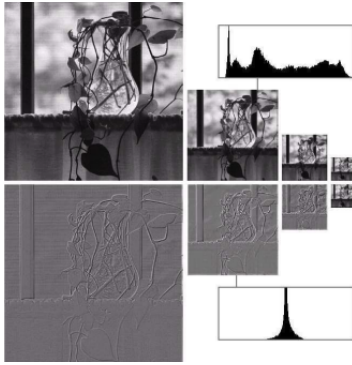
If the intrinsic image blur σ_{image} is known, σ_{init} the extra blur required to reach σ_0 can be therefore calculated from:

$$\sigma_{init} = \sqrt{\sigma_0^2 - \sigma_{image}^2}$$

So having an initial level of blurring by σ_{init} might seem redundant, but it allows us to tune the remaining filter σ_c parameters to be the same for each level.

Therefore σ_{init} allows us to add an additional degree of blur to the intrinsic blur in the image to generate a regular pyramid structure blur σ_0 at each level.

It also allows to generate any arbitrary degree of blur for each level, useful if we want to compute Gaussian derivatives at each level.



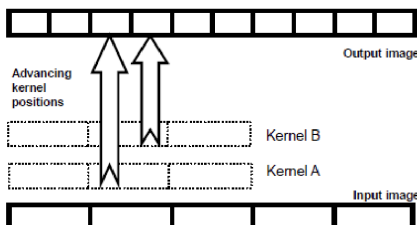
At the top is a Gaussian pyramid and associated histogram of the top level, below is a DOG pyramid and associated histogram of the top level.

Notice the difference between the structure of the respective histograms, the top version has the full image grey-level structure present.

The bottom histogram shows only the distribution of highest spatial frequency second order gradients (zero mean)

8.3.4 Expanding Pyramid Levels

To expand a Gaussian interpolation, use two Gaussian kernels A & B applied to the same window in the input image, but their Gaussian functions are displaced by $\pm$ half a pixel with respect to each other. The outputs from these kernels fill two consecutive output pixels.



Since the current level of blur σ_i is combined with the expansion σ_e the total blur σ_{exp} becomes :

$$\sigma_{exp} = \rho \sqrt{\sigma_i^2 - \sigma_e^2}$$

The above example illustrates a factor of 2 expansion, illustrating how disparity maps can be expanded between pyramid levels.

8.3.5 DOG and LOG Pyramids

The DOG pyramid can be created from the Gaussian pyramid:

1. Convolve current level with a Gaussian filter
2. Sub-sample
3. Apply a second Gaussian convolution with $\sigma = \rho \sqrt{\sigma_i^2 - \sigma_e^2}$ to obtain a second image now blurred with σ value 1.6 times that of the blur for the current level (this process will be the same for all levels)
4. Subtract this additionally blurred level from the original level, i.e. compute the Difference of Gaussians filtered images
5. Repeat for next level as before

$$DOG(I_i(x, y)) = I_i(x, y) - G_\sigma(I_i(x, y)).$$

Some approaches to the above involve expansion and subtraction of pyramid layers, however these approaches contain intrinsic inaccuracies.

A LOG pyramid can be constructed similarly by computing the second order partial derivatives (horizontal and vertical) at each level of the Gaussian pyramid and then adding these together to form each new level.

8.4 Pyramid Uses

- Multi-resolution image processing and efficient scale-space tracing
- Anti-aliased (non-pixelated) image zoom-in and zoom-out
- Multi-resolution preview structure for above:
 - View images at any octave resolution - penalty of 1/3 increase in storage for octave pyramid.
 - Apply image operations at low resolution and see results quickly while full resolution computed in background
- Possible to express many image analysis algorithms in terms of pyramids to get scale-independent processing,
 1. At each level of the pyramid apply the *same* operator, e.g. run a kernel to enhance a specific pattern and detect instances
 2. Starting at the coarsest level detect instances of the pattern
 3. At the next level search refine the previous estimate by searching about the location found in the previous level and detect new instances.
 4. The above process is good for simple patterns such as edge segments.

In general, search over a pyramid is a two step process.

1. Apply the same detection algorithm at all levels
2. For each instance found at each level refine solution at finer levels. Refinement search is now highly constrained, e.g. if the expansion factor between levels is 2 we only need to search 5 pixels in the next level

First principled pyramids by Burt and Adelson in early 1980s for image coding using LOG pyramid - they also proposed seamless image splining:

- Merge two images without a join using pyramids
- Form DOG pyramids for each image
- Average each pyramid level over a constant N pixels, N usually = 1
- Reassemble the pyramid

No visible seam as each scale averaged over an appropriate spatial distance of N for each scale. DOG pyramid is the basis for the SIFT recognition algorithm

8.4.1 Pyramidal Seamless Splining

Far left input images are converted to pyramids, as is the mask image (half-white, half-black). The mask pyramid and its inverse are used to select opposite halves of the input images. These halves are averaged by 1 pixel overall at each level and expanded to form a complete image

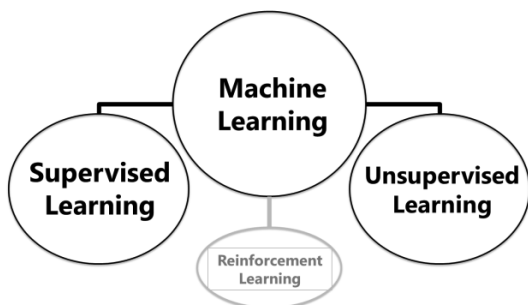


9 Introduction to Machine Learning

ML is slightly distinct from statistics, which is traditionally concerned with making inferences from data, e.g. determining if an effect is present in a scientific study. Instead, machine learning focuses on trying to estimate the value of unknown variables given some data which might predict them.

Machine learning (ML) makes minimal assumptions about the data generating systems. Classical statistics make assumptions about the model that fits the data and vary considerably from ML in terms of computational tractability. Classical statistics normally can deal only with a few dozen variables and sample sizes are relatively small.

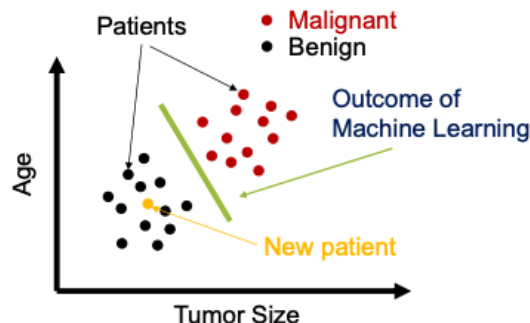
Machine learning is a subset of artificial intelligence - learn automatically and improve from experience



9.1 Supervised Learning

In supervised learning, the algorithm is trained on a labelled dataset, each training example has both the input data (features) and the correct answer (label). The goal is for the model to learn a general rule that maps inputs to outputs.

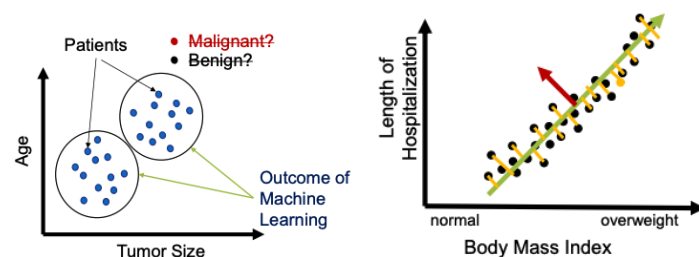
Example We might use supervised learning to predict if a patient's tumor is malignant or benign



Here we are using labelled data for supervised learning, and getting a categorical predicted variable. This is what we call **Classification**. If we were to predict continuous variables, e.g. predicting length of hospitalisation, it would be **regression**

9.2 Unsupervised Learning

In unsupervised learning, there are no explicit labels in the training data. The model tries to discover patterns or structures directly from the data. Used for tasks like clustering, referring to grouping together similar data



Examples Left is the Tumor example, this time with clustering. On the right, is an unsupervised learning example to discover an unknown structure using dimensionality reduction and Principle Component Analysis.

Supervised Learning	Unsupervised Learning
Learning to approximate reference labels/answers	Learning underline data structure
Requires correct labels	No labels or feedback required
Model does not affect input data	Model does not affect input data

9.3 K Nearest Neighbors (KNN)

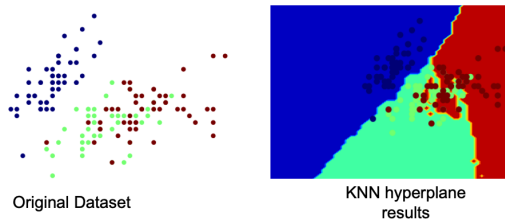
An algorithm for supervised learning (classification/regression) Its distance metrics are:

$$d(x, y) = \sqrt{\sum_{i=1}^n (y_i - x_i)^2}$$

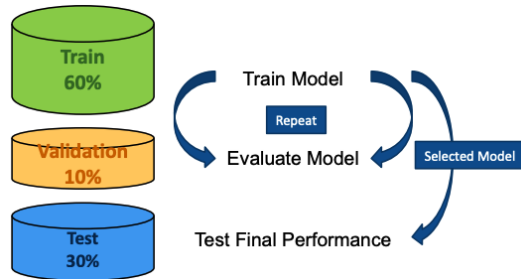
Euclidean Distance

$$d(x, y) = \sum_{i=1}^n |y_i - x_i|$$

Manhattan Distance



9.4 Performance Evaluation

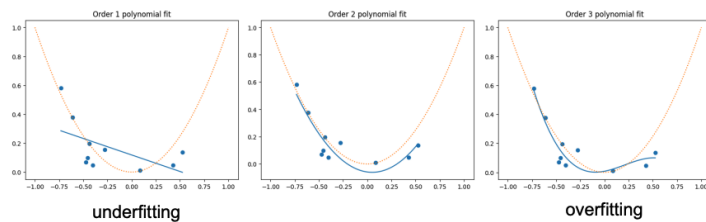


9.4.1 Overfitting and Underfitting

Overfitting High variance, the model is very flexible and learns irrelevant characteristics on training data. It performs poorly on testing data, performance drops off after some epochs.

Underfitting High bias, the model cannot capture the complexity of the data (few parameters).

Example Polynomial fitting: $y = a_0x^0 + a_1x^1 + \dots + a_qx^q$



9.5 K-Fold Cross Validation



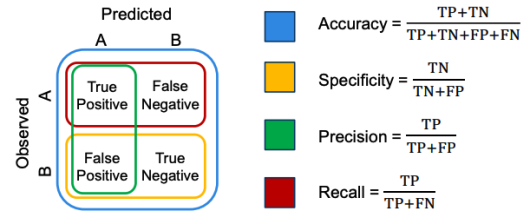
Cross-validation is a resampling procedure used to evaluate machine learning models. One key advantages is that the testing samples are independent between folds. There is a need to control the class' distributions of the training/testing data

9.6 Subsampling Methods

- Hold-Out method
- Simple Resampling Methods
 - K-fold cross validation

- Stratified K-fold cross validation
- Leave one out cross validation
- Multiple Resampling Methods
 - Nested k-fold cross validation
 - Random subsampling
 - Bootstrap sampling
 - Permutation testing

9.7 Performance Metrics



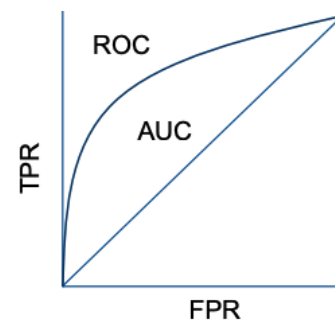
9.7.1 F1-Score

$$F1 = \frac{2}{\frac{1}{recall} + \frac{1}{precision}} = 2 * \frac{precision * recall}{precision + recall}$$

The F1 score is a way of combining the precision and recall of the model, and it is defined as the harmonic mean of the model's precision and recall.

9.7.2 Receiver Operating Characteristics

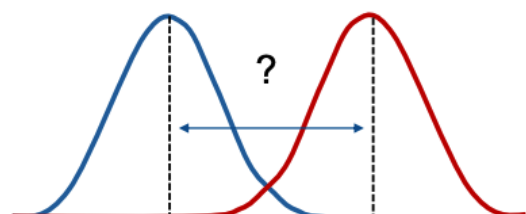
ROC curve is a performance measurement for the classification problems at various threshold settings. ROC is a **probability curve** and area under the curve (AUC) represents the degree or **measure of separability**. The ROC curve is plotted with TPR against the FPR where TPR is on the y-axis and the FPR is on the x-axis.



9.7.3 Comparing the Performance of Algorithms

Null-Hypothesis Statistical Testing for algorithm evaluation. The t-test is a statistical hypothesis test in which the test statistic follows a t-distribution under the null hypothesis.

The common assumptions: independence of measurements, normality, adequate sample size, and equality of variance in standard deviation.



9.7.4 Non-Parametric Tests

The Wilcoxon signed-rank test is a general test to compare distributions in paired samples. This test is usually the preferred alternative to the paired t-test when the assumptions are not satisfied.

It determines if the two populations seem to be the same or different based upon the ranks of the absolute differences. Ranking procedures are commonly used in non-parametric methods as this moderates the effect of any outliers.

9.8 Image Classification Example

Problem: Train an algorithm to identify whether images were taken during the day or night.



For any given classification problems we need to prepare our input data to the algorithm: features, labels, (X,Y). this is a binary classification problem (labels are 0,1). What is our feature vector X?

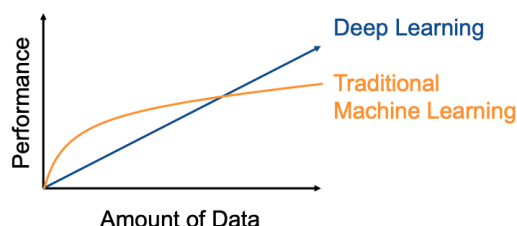


These histograms represent the distribution of colours in the image (8 levels for 3 colour channels, 512 features)

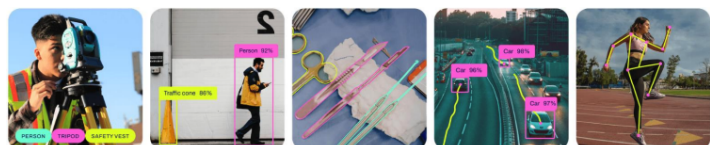
10 Introduction to Deep Learning

The problem we want to solve here? avoiding feature engineering.

Deep Learning mimics how brain neurons communicate and learn. Allows a high level of abstraction in learning. Eliminates the need for handcrafted features. State-of-the-art in image recognition. High demand in data and computational power.



10.1 Uses in Computer Vision

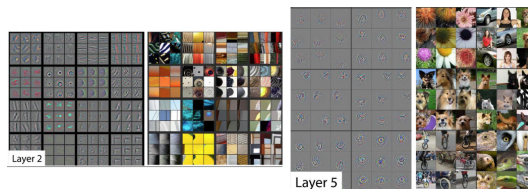


classifying/detecting/segmenting/tracking objects, pose estimation

10.2 Avoiding Feature Engineering

Unlike other learning approaches, deep learning does not require you to select, transform, create, or otherwise format input variables from raw data to improve its performance.

Deep learning can learn directly from raw input, without human intervention, as can be seen below;



This doesn't mean we can't or shouldn't feature engineer, in some cases it can be beneficial, but the benefit of deep learning is that you **don't have to**.

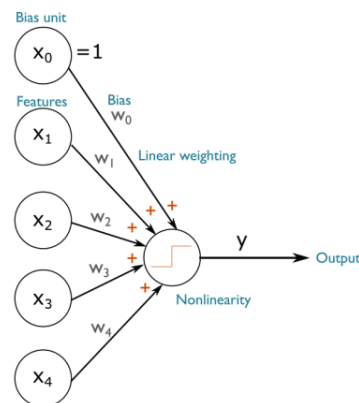
10.3 The Perceptron

Perceptrons are inspired from how brain neurons work, input is connected to the output via an activation function. It can learn to classify linearly separable patterns.

$$y(x) = F_A(w^T x)$$

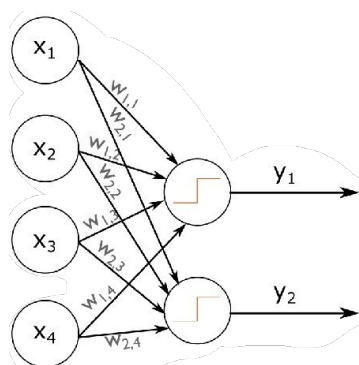
The bias term introduces an offset to the function, bias term is always equal to 1.

$$\begin{aligned} y(x) &= F_A(w_o x_o + w_1 x_1 + \dots w_n x_n) \\ &= F_A(w_o + w_1 x_1 + \dots w_n x_n) \end{aligned}$$



Considering multiple outputs...

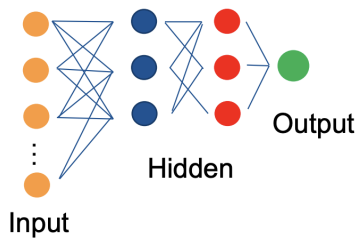
$$\begin{aligned} y_1(x) &= F_{A_1}(w_{1,1}, x_1 + w_{1,2}, x_2 + \dots w_{1,n}, x_n) \\ y_2(x) &= F_{A_2}(w_{2,1}, x_1 + w_{2,2}, x_2 + \dots w_{2,n}, x_n) \\ y_m(x) &= F_{A_m}(w_{m,1}, x_1 + w_{m,2}, x_2 + \dots w_{m,n}, x_n) \end{aligned}$$



10.3.1 Multi-Layer Perceptron

A neural network multi-layer perceptron (NNLMP) is a feed forward neural network. They consist of minimum three layers of nodes, which are input, hidden, and output layer.

$$\begin{aligned} y^L &= f^L(W^L X^L) \\ y^L &= f^L(W^L f^{L-1}(W^{L-1} X^{L-1})) \\ y^L &= f^L(W^L f^{L-1}(W^{L-1} f^{L-2}(W^{L-2} X^{L-2}))) \end{aligned}$$



Each neuron in a connected layer receives an input from all the neurons present in the previous layer, thus they're densely connected.

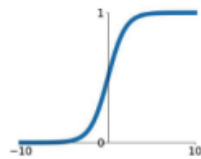
In other words, the **dense layer is a fully connected layer**, meaning all the neurons in a layer are connected to those in the previous layer.

10.4 Activation Functions

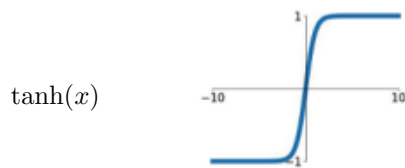
Internal nodes of a network are processing functions. Activation functions are required to be differentiable functions. Small changes in the weights results in a small change in the output. Activation functions are monotonic but their derivatives are not.

Sigmoid The **sigmoid activation function**, also called the logistic function, is a very popular activation function for neural networks. The input to the function is transformed into a value between 0.0 and 1.0.

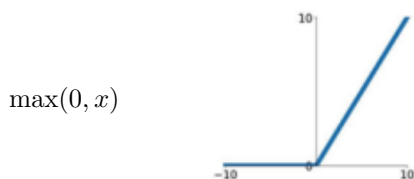
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



tanh The advantage of the **Hyperbolic Tangent (tanh) function** is that it produces a zero centred output, thereby supporting the back propagation process. The tanh function has been mostly used in recurrent neural networks.

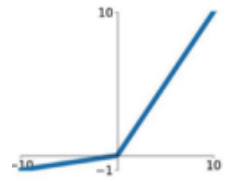


ReLU ReLU stands for **Rectified Linear Unit** and is a type of activation function. Mathematically, it is defined as $y = \max(0, x)$. It's a simple function which allows models to account for non-linearities and interactions.



Leaky ReLU An improved version of the ReLU activation function. For the ReLU activation function, the gradient is 0 for the values of inputs that are less than zero, which would deactivate the neurons in that region and may cause dying ReLU problems.

$$\max(0.1x, x)$$

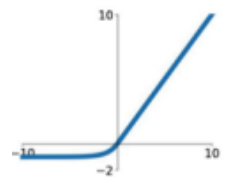


The Maxout Unit A generalisation of the ReLU and the leaky ReLU functions. It is a piecewise linear function that returns the maximum of the inputs. Both ReLU and leaky ReLU are special cases of Maxout.

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU Exponential Linear Unit or its widely known name ELU is a function that tends to converge cost to zero faster and produce more accurate results. Different to other activation functions, ELU has an extra alpha constant which should be a positive number

$$\begin{cases} x & x \geq 0 \\ \sigma(e^x - 1) & x < 0 \end{cases}$$



Softmax Function Useful when the total sum of the output of a layer needs to be equal to one. It is used to produce a vector of probabilities and it is common in the last layer of deep neural networks for multi-class classification.

$$y_i = \frac{e^{x_i}}{\sum_{j=1}^J e^{x_j}}$$

Hierarchical Softmax Useful when the number of classes is high, and it is computationally expensive to use the softmax function. It approximates probability distributions as a binary tree with larger depth corresponding to less probable class activations.

10.5 Training an MLP

A Multi-Layer Perceptron can be trained by:

- Forward Propagation
- Error Computation
- Back Propagation
- Parameter Update

10.5.1 Forward Propagation

$$f(x) = \frac{1}{1 + e^{-x}}$$

Activation Function

$$h_1 = f(W_1 x + b_1)$$

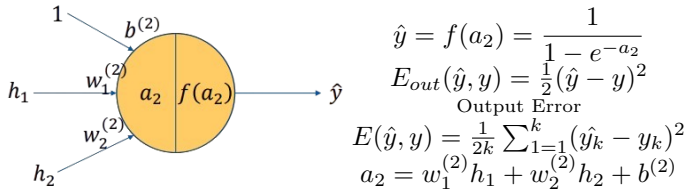
$$h_2 = f(W_2 h_1 + b_2)$$

$$\vdots$$

$$\hat{y} = h_n$$

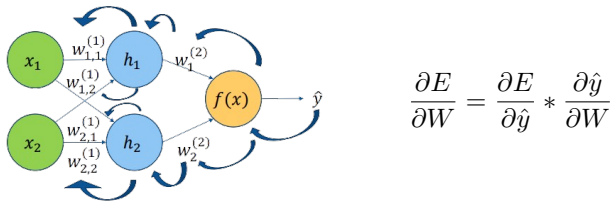
The first step, where data flows from the input layer through the hidden layer(s) to the output. The network takes the input features, applies weights and activation function, and produces a prediction, following the above equations.

10.5.2 Error Computation



At this stage, the network calculates how far off the prediction is from the true target value. Typically involving a loss function like mean squared error for regression or cross entropy loss for classification. The goal here is to quantify the networks current performance on training data.

10.5.3 Back propagation

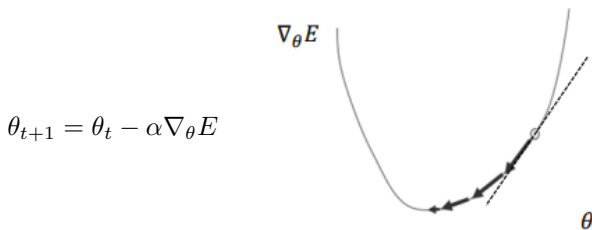


This is a critical stage that calculates how each weight in the network contributed to the error. It works backwards from the output to the input, hence the name. From this we can find out how much each weight should change to reduce the error.

10.6 Optimisation Process

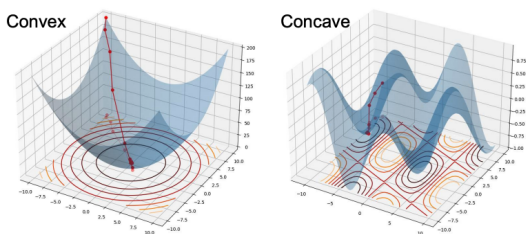
First, we choose hyperparameters. In a Multi-Layer Perceptron, these normally are learning rate, layer size, and regularisation constant. Next is weight initialisation.

Grid search is a basic hyperparameter tuning method. It builds a model for each possible combination of all the hyperparameter values, it evaluates each model, then selects the architecture which produces the best results.



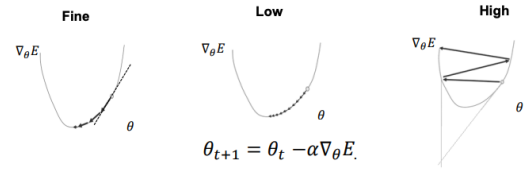
Gradient descent minimises the loss function iteratively:

- Computes the slope (gradient): first order derivative of the function at current point
- Move in the opposite direction of the slope increase from the current point.



Stochastic Gradient descent:

- Take each sample
- Feed it to Neural network
- Calculate its gradient
- Use the gradient we calculated in previous step to update weights



A small learning rate slows down training and might be prohibitive in large models. A large learning rate causes large parameter updates that may cause divergence problems.

10.6.1 Types of Loss Function

Loss functions quantify how well the model is performing between predicted ($\hat{y}$) and actual (y) values. The choice of function is heavily dependent on your task, i.e. regression or classification, and loss functions have different mathematical properties that make them suitable in different scenarios. In all of the following cases, k is our batch size.

Mean Squared (L_2) Error

$$E(\hat{y}, y) = \frac{1}{k} \sum_{k=1}^k (\hat{y}_k - y_k)^2$$

Best for regression problems where outliers should be heavily penalized.

Mean Absolute (L_1) Error

$$E(\hat{y}, y) = \frac{1}{k} \sum_{k=1}^k |\hat{y}_k - y_k|$$

Also for regression problems, but more robust to outliers. Penalises all errors linearly, not quadratically like MSE.

Negative Log Likelihood / Cross Entropy Loss

$$E(\hat{y}, y) = -\frac{1}{k} \sum_{k=1}^k (y_k \log(\hat{y}_k) - (1 - y_k) \log(1 - \hat{y}_k))$$

The standard for binary and multi-class classification problems. Measures the difference between probability distributions.

Hinge Loss

$$E(\hat{y}, y) = \frac{1}{k} \sum_{k=1}^k \max(0, 1 - y_k \hat{y}_k)$$

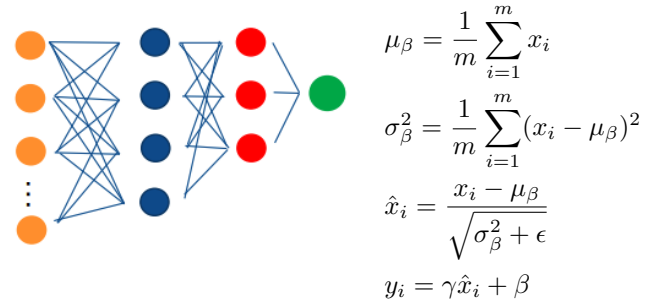
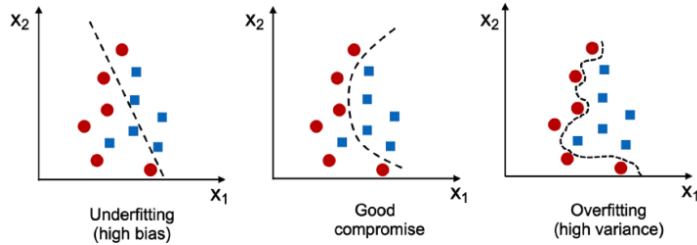
Primarily used for maximum margin classifiers like support vector machines. penalises incorrect and unconfident predictions.

Kullback-Leibler (KL) Loss

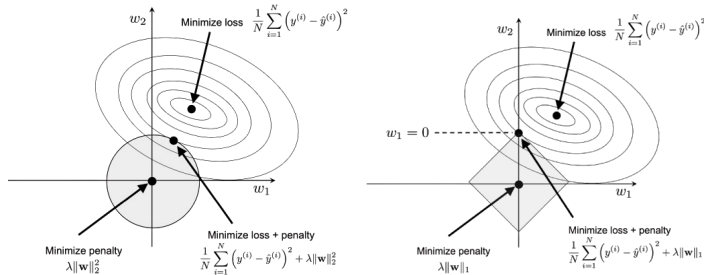
$$E(\hat{y}, y) = \frac{1}{k} \sum_{i=1}^k D_{KL}(y_i || \hat{y}_i) = \frac{1}{k} \sum_{i=1}^k (y_i \log(y_i)) - \frac{1}{k} \sum_{i=1}^k (y_i \log(\hat{y}_i))$$

Used in probabilistic models working with distributions, measures how one distribution diverges from another.

10.7 Regularisation



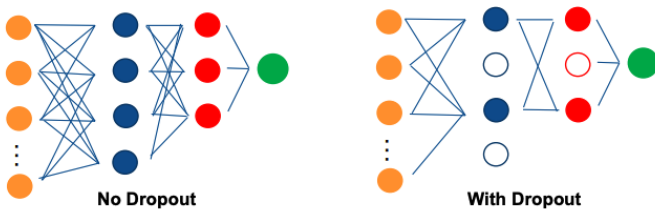
10.7.1 L1 & L2 Regularisation



left: L2 Regularisation, right: L1 Regularisation

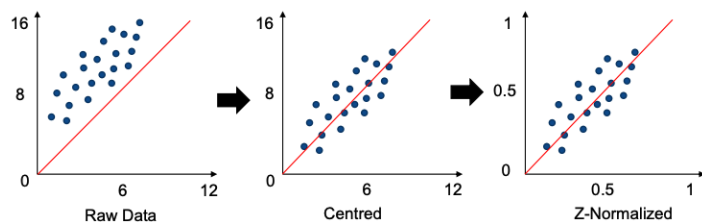
10.7.2 Dropout

The dropout layer is a mask that nullifies the contribution of some neurons towards the next layer and leaves all others unmodified. It can be applied to the input vector, in which case it **nullifies some of its features**, but we can also apply it to a hidden layer, in which case it **nullifies some hidden neurons**.



10.8 Normalisation

Normalisation of the input data is a standard process for machine learning models. Normalisation involves removing the mean and dividing by the standard deviation. It deduces the amount of weights the neural network needs to shift.



10.8.1 Regularisation - Batch Normalisation

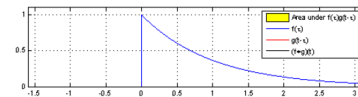
Batch normalisation is a technique to normalise the output of each layer of a network. It estimates a moving average and variance during training.

Batch normalisation accelerates training, as it reduces convergence time. The mean and standard deviation are calculated over the mini batches and γ and β are learnable parameters.

11 Convolutional Neural Networks

11.1 Convolution vs Cross Correlation

Convolution is a mathematical operation



Continuous Domain:

$$y(t) = (f * g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t - \tau)d\tau$$

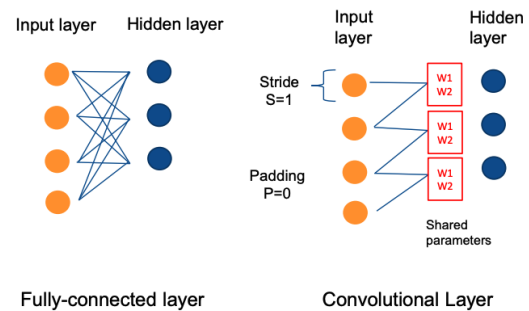
Discrete domain:

$$y(t) = (f * g)(t) = \sum f(\tau)g(t - \tau)$$

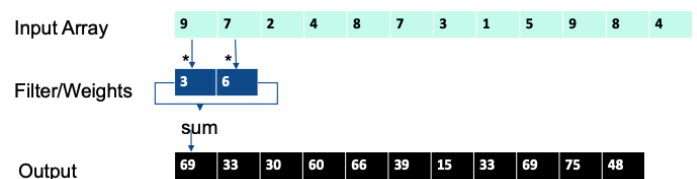
Cross Correlation:

$$y(t) = f \otimes g(t) = \sum g(\tau)g(t + \tau)$$

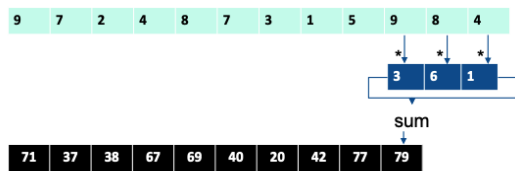
11.2 Convolutional Neural Network vs MLP



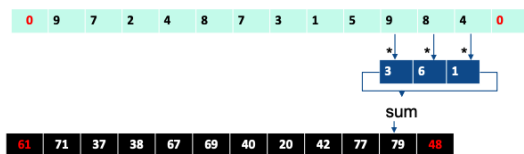
11.2.1 Filter, Stride, Padding Example



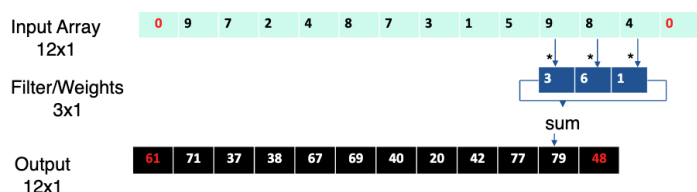
Here, we have a 1x12 filter and we are filtering with a length-2 kernel, sliding the filter one step at a time (i.e. stride=1), multiplying elements over the filter weights and summing them to yield the output. $[9 \cdot 3 + 7 \cdot 6, 7 \cdot 3 + 2 \cdot 6, \dots, 9 \cdot 3 + 8 \cdot 6]$.



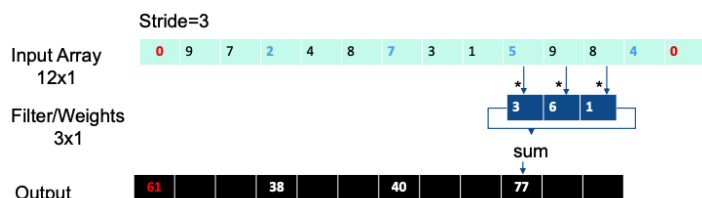
Here we change the filter to length 3, keeping everything from before the same. Where the output length was 11 before, it is now 10. Filter size directly affects the size of the feature map and receptive field



To preserve input length, we can add a 0 at each end, so that with filter length 3 and padding 1, we retain the original output length of 12. Padding controls spatial dimensions of the output and helps retain edge information



Changing stride to 3, the filter jumps in steps of 3 instead of 1. As a result, there will only be 4 outputs, demonstrating how stride controls down-sampling of feature maps, reducing computational load, and capturing coarser structures



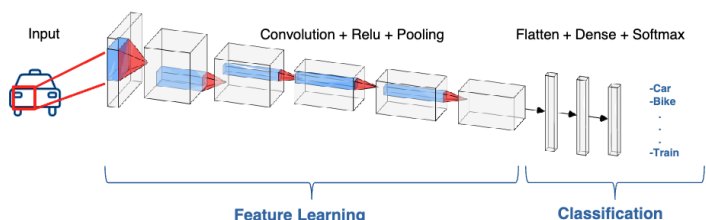
The output parameters is defined by:

$$\frac{N - F + 2P}{S} + 1$$

Where N = input size, F = filter size, P = padding, S = stride. This lets us predict feature-map dimensions before training.

11.3 Convolutional Neural Networks

The model learns to extract a hierarchy of features. CNN has applied successfully both in 1D and 2D classification.

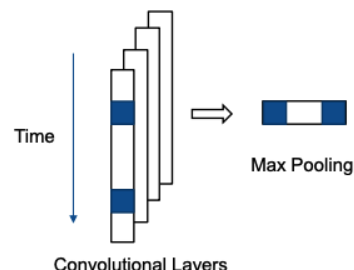


11.3.1 Model Layers: Pooling

Pooling layers are used for downsampling. It reduces the number of parameters to learn, reducing overfitting. It can also result in invariance.

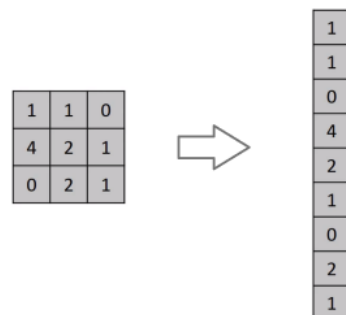
- Max Pooling - picks the strongest activation, helps capture prominent features
- Average Pooling - captures the mean, smooths response
- L2 Norm Pooling
- Stochastic Pooling
- Spectral Pooling

Pooling reduces parameter count, limits overfitting, and introduces a degree of spatial invariance.

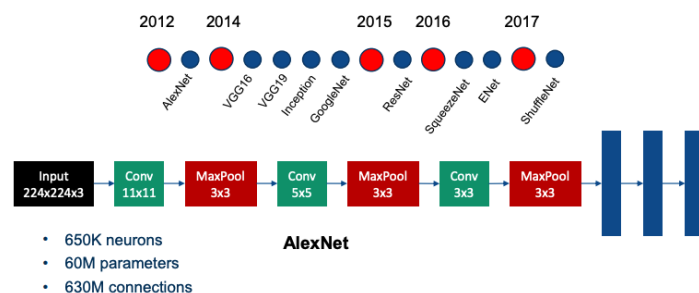


11.3.2 Model Layers: Flatten

Flattening is converting the data into a 1-dimensional array for inputting it to the next layer. We flatten the output of the convolution layers to create a single long feature vector. It is connected to the final classification model, which is called a fully-connected layer.



11.3.3 Classic CNN Architecture



11.4 Summary

CNNs have been proven very effective in computer vision and it is one of the most successful applications of Deep Learning.

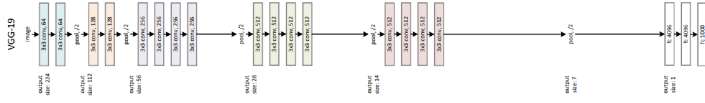
CNNs are more efficient than a MLP architecture which is fully connected.

They exploit the fact of extracting local features in a hierarchical way. Training in CNN can be more challenging than MLP

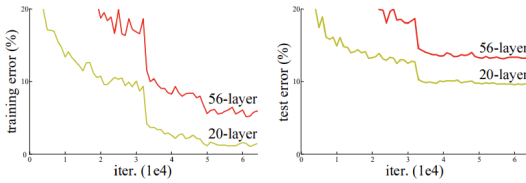
12 Other Feed Forward Deep Neural Networks

12.1 Residual Network

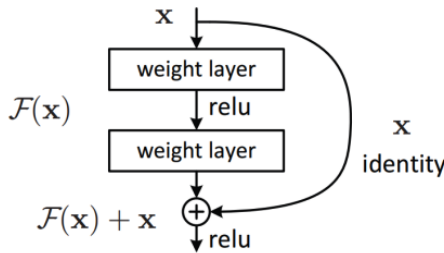
The VGG-19 network architecture. A sequence of 3x3 convolutional layers, with pooling operations to reduce spatial dimensions. The output dimensions shrink at each stage.



Residual Networks are designed to solve a particular problem - the degradation problem - deeper networks, in theory, should perform better but practically become harder to optimise. We can see this in the following graphs of training and test errors



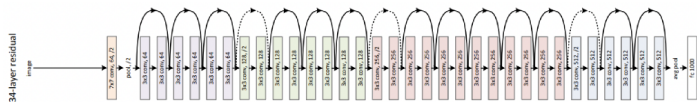
As we can see, the deeper 56 layer network, in red, shows higher training and test error than the shallower 20 layer network. The deeper network might, for example, be a plain network of many convolution layers without the pooling operations.



This is the core structure of residual networks. An input x follows two paths,

- A traditional path through weight layers with ReLU activations (producing $F(x)$)
- A *shortcut*, or *skip connection* that bypasses these layers (identity x)

These paths are combined by the addition $F(x) + x$, and the final output is passed through a ReLU activation. These residual connections are put throughout our residual network as such.

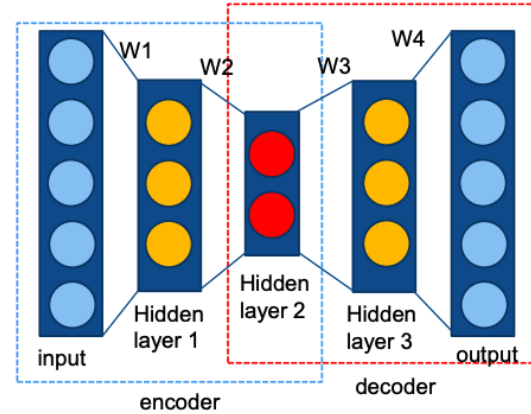


In this diagram, a solid arc indicates *pure identity*, the $F(x) + x$ from before, and a dotted arc indicates *projection*. At these blocks, the resolution is changing, e.g. from 64 to 128, so instead of adding x to $F(x)$, we run x through a 1x1 convolution to match dimensions. This is expressed as $F(x) + W_s x$

The key idea is that instead of directly learning the desired mapping $H(x)$ the network learns the residual mapping $F(x) = H(x) - x$. This makes it easier for the network to

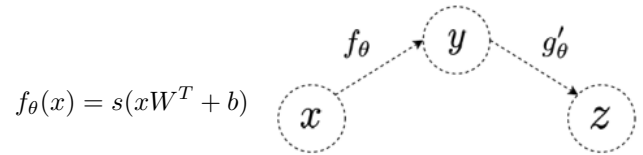
learn identity mappings when needed, which helps the gradient flow during backpropagation.

12.2 Autoencoders



An autoencoder is a type of neural network design to learn efficient data representations unsupervised by trying to reconstruct its input.

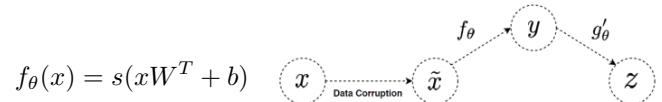
12.2.1 Autoencoder



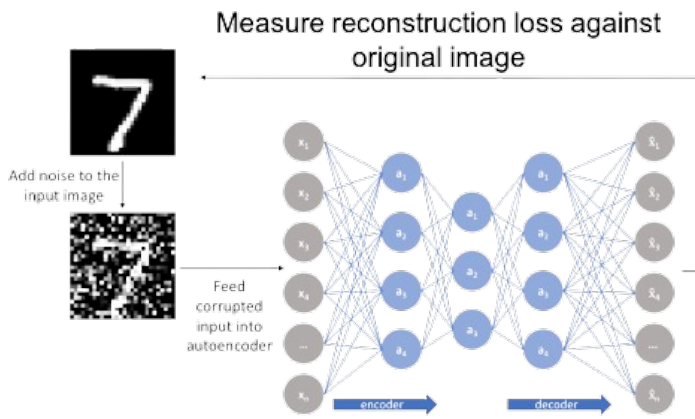
The autoencoder training process consists of a forward pass, then loss computation, then backpropagation to update weights. The network learns to minimise reconstruction error, typically using mean squared error loss for continuous data.

12.2.2 Denoising Autoencoder

Denoising autoencoders are a variation where a corrupted or noisy version of the input data is fed in, and the network is trained to recover the original. This forces the network to learn robust features that can ignore noise and capture true data structure. The network develops more robust internal representations and can generalise better to new data

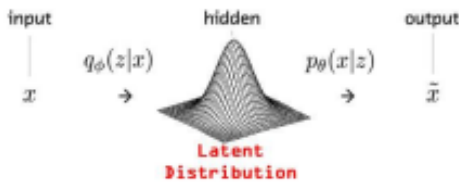


To denoise an image the autoencoder architecture doesn't change, but the training method does. Noise is added to the input image, but the loss function now compares the original image to the reconstructed image. It works and can be used in many other contexts as well.

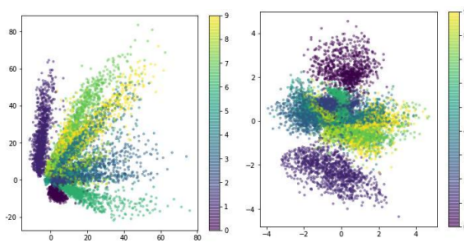


12.2.3 Variational Autoencoder

Variational autoencoders take a probabilistic twist. Instead of encoding inputs as a single point in latent space, VAEs encode them as a probability distribution. During training, points are randomly assigned from these distributions. This adds regularisation and creates a continuous meaningful latent space. VAEs include a term in their loss function that pushes the latent distribution towards a normal distribution

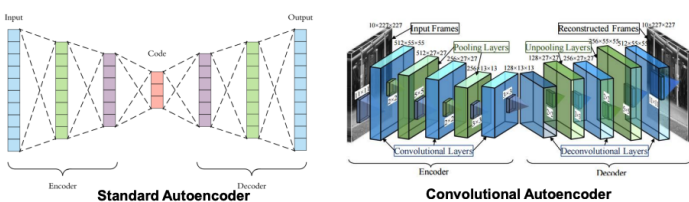


Encodings of variational autoencoders trained on all MNIST digits. Colour bars pertain to MNIST values



Left would imply that there is more intra-class variation in the 1s than there is inter-class variation between the 1s and 0s. Is this a sensible subspace? On the right, we see that 1s and 0s are most different, that 9s and 7s are dissimilar, and more similar to 1s and 0s.

12.2.4 Convolutional Autoencoder



The standard autoencoder comprises fully connected layers (as shown left). The convolutional autoencoder comprises mirrored convolution layers coupled by max pooling and joined by fully connected bottleneck "code" layers

12.2.5 Architectural Choices

There are some architectural choices to make regarding network structure and parameters for autoencoders, namely;

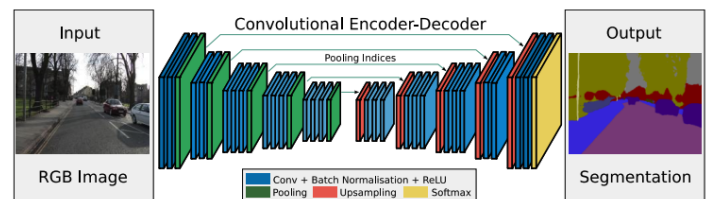
- Number of layers - deeper networks can learn more complex representations, but are harder to train.
- Overcomplete representation - when hidden layers are larger than inputs, regularisation becomes crucial to prevent identity mapping
- Activation functions - different options affect representation capabilities
- Loss function - different losses optimise for different reconstruction properties

The regularisation technique chosen also has an affect, for example, l2 penalises large weights to prevent overfitting, batch normalises layer input to stabilise and accelerate training, and dropout randomly deactivates neurons during training to create redundancy.

12.3 Image Segmentation

12.3.1 SegNet

SegNet architecture is based on VGG16 network architecture, however, there are no fully connected layers and hence it is only convolutional. Key components of SegNet is the decoder network which consists of a hierarchy of decoders one corresponding to each encoder.



SegNet architecture reuses max-pooling indices in the decoding process and has several practical advantages. It improves boundary delineation, reduces the number of parameters enabling end-to-end training, and this form of up-sampling can be incorporated into any encoder-decoder architecture.

12.3.2 UNet

UNet Architecture - An (almost) symmetric network that was designed for segmenting biomedical images, has no fully connected layer.

Uses the same feature maps that are used for contraction to expand a vector to a segmented image. Up sampling path has a large number of feature filters

Special weighting uses morphology operations to enhance learning borders

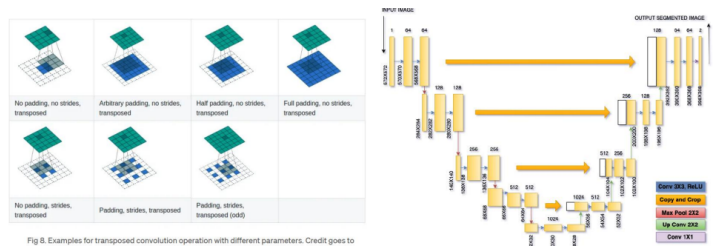


Fig 8. Examples for transposed convolution operation with different parameters. Credit goes to https://github.com/ydmoulin/conv_architectures [10]

13 Time Dependant DNNs

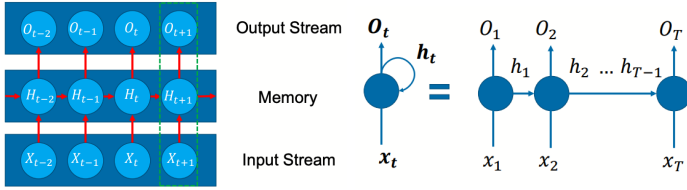
13.1 Deep Recurrent Networks

Recurrent networks have a memory and as a result can remember previous states, unlike pure feed-forward networks. State memory allows recurrent networks to perform general processing and manipulation of sequences, such as:

- Classify input Sequence
- Predict the next element of an input sequence
- Learn structure in input sequences
- Translate sequences
- Video understanding

13.1.1 Recurrent Neural Networks

RNNs accumulate information from each time step, they can learn representations for variable length sequences



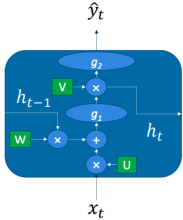
$$o_t = f(x_t, h_{t-1})$$

$$h_t = o_t$$

$$h_t = f(Ux_t + Wh_{t-1})$$

13.1.2 Forward/Back Propagation in RNNs

Forward

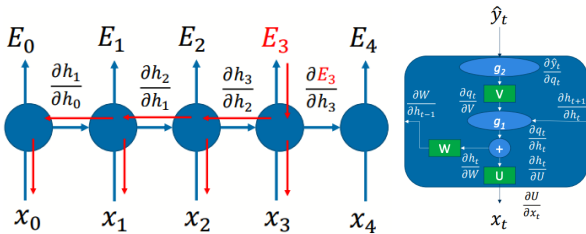


$$h_t = g_1(Ux_t + Wh_{t-1})$$

$$\hat{y}_t = g_2(Vh_t)$$

g_1, g_2 are activation functions

Back



E_t is the error(loss) at time t, i.e. the difference between the output at t and the ground truth

recursive weights:

$$\frac{\partial E_t}{\partial W} = \frac{\partial E_t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial q_t} \frac{\partial q_t}{\partial h_t} \frac{\partial h_t}{\partial W}$$

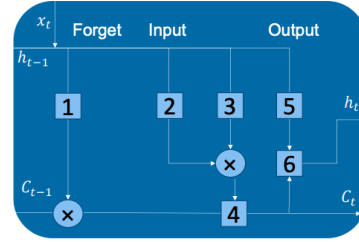
$$\frac{\partial h_t}{\partial W} \rightarrow \frac{\partial h_t}{\partial W} + \frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial W} \rightarrow \dots$$

This comes with some problems, namely:

- Vanishing Gradient
 - Gradient becomes too small over time, and network becomes incapable of learning as no change can be made
- Exploding Gradient
 - Gradient becomes too big over time, and network becomes very unstable.

The solution to this is gradient clipping, to limit the gradient

13.2 Long Short-Term Memory (LSTM)



C - cell state (Long term memory), h - hidden state (Short term memory)

To solve vanishing/exploding gradient, we introduce input, output, and forget gates.

$$[1] \quad f_t = \sigma(W_f * [h_{t-1}, x_t] + b_f)$$

$$[2] \quad i_t = \sigma(W_i * [h_{t-1}, x_t] + b_i)$$

$$[3] \quad \tilde{C}_t = \tanh(W_C * [h_{t-1}, x_t] + b_C)$$

$$[4] \quad C_t = (f_t * C_{t-1} + i_t * \tilde{C}_t)$$

$$[5] \quad o_t = \sigma(W_o * [h_{t-1}, x_t] + b_o)$$

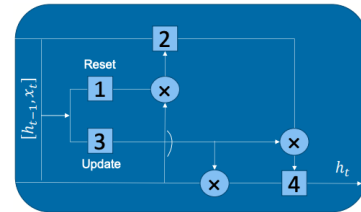
$$[6] \quad h_t = o_t * \tanh(C_t)$$

Forget gate : Decide how much of the previous data will be used/forbidden, with a value between 0 and 1.

Input gate : Decide what input information is relevant for updating the cell state

13.3 Gated Recurrent Unit

Key difference with LSTM: The forget and input gates are replaced by reset and update gates, and there is no output gate.



$$[1] \quad r_t = \sigma(W_r * [h_{t-1}, x_t])$$

$$[2] \quad \tilde{h}_t = \tanh(W * [r_t * h_{t-1}, x_t])$$

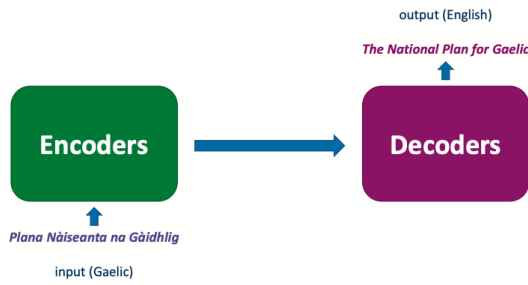
$$[3] \quad z_t = \sigma(W_z * [h_{t-1}, x_t])$$

$$[4] \quad h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

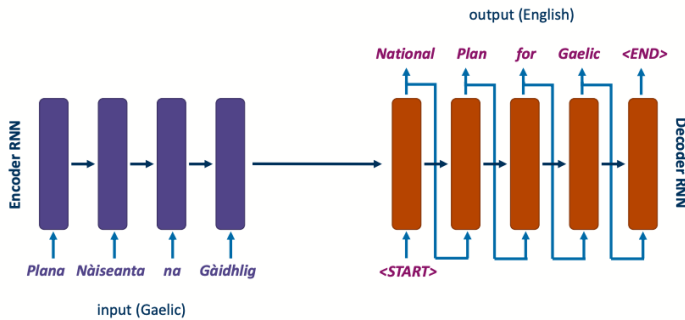
Reset gate : Decide how much of the previous data will be forgotten.

Update gate : Decide how much of the previous data will be used for updating the hidden state

13.4 Machine Translation



13.4.1 Using RNN

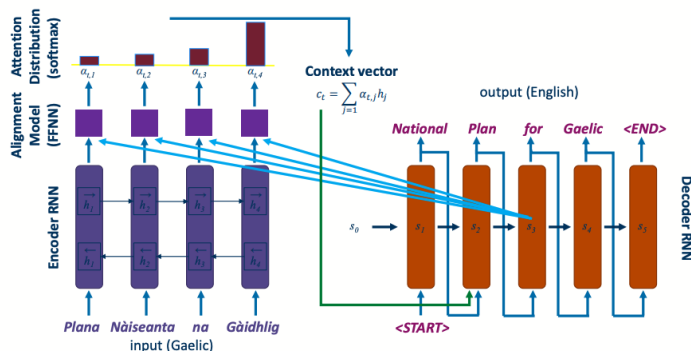


Problems: Can not remember long term dependencies due to vanishing gradient, not effective for handling long sequences. Also sequential computation is time consuming because of the lack of parallelisation

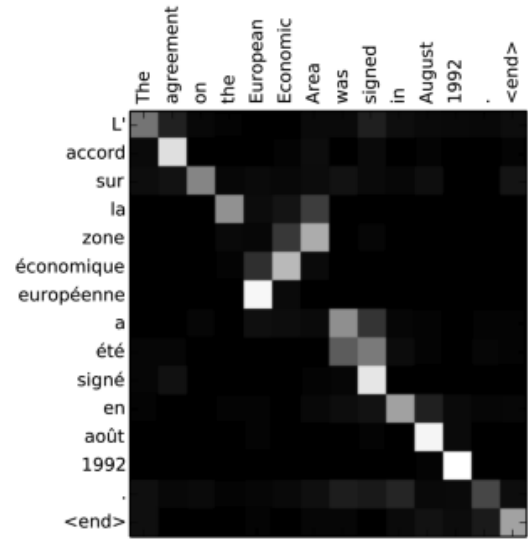
13.4.2 Using RNN with Attention

Given the hidden state s_{t-1} (for the previous target word y_{t-1} as in the RNN decoder in the example) the **current** target word y_t can be computed by:

- Compute scores (similarities or weights) between s_{t-1} and each h_j (representation of each word in the source sentence in the RNN encoder in the example)
- Normalise the scores computed above using softmax
- Get the context vector c_t as a weighted sum of the representations of the words in the source sentence, i.e. $c_t = \sum_{j=1} \alpha_{t,j} h_j$
- The hidden state $s_t = f(s_{t-1}, y_{t-1}, c_t)$ will be used for generating the current target word y_t



Advantages: improving performance in a wide range of applications, ingratiating the gradient vanishing problem, and the attention distribution provides interpretability to the learned model



13.5 Transformers

Unlike RNNs, sequential computation is not needed. "entirely on self-attention to compute representations of its input and output" (Vaswani et al.)

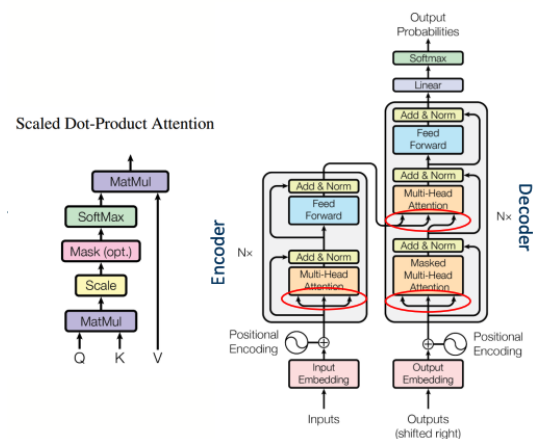
"Self-attention, sometimes called intra-attention is an attention mechanism relating different positions of a single sequence in order to compute a representation of the sequence" (Vaswani et al.)

Allows more parallelisation, requires less time to train.

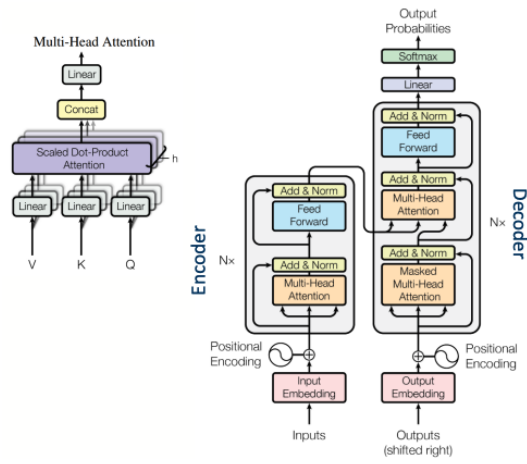
13.5.1 Attention

Mapping a query and a set of key-value pairs to an output:

- Encoding each word in the sentence (as an example) as query, key, and value, respectively.
- Computing the attention by dot-product between Q and K: $\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$

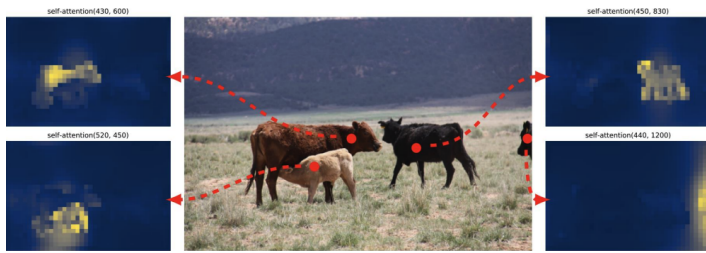


Multi-Head Attention learning different sets of linear projections for query, key, and value. Allows the model to jointly attend to information from different representation subspaces, e.g. shorter term vs longer term dependencies. Positive Encoding: represent the location of items in a sequence.

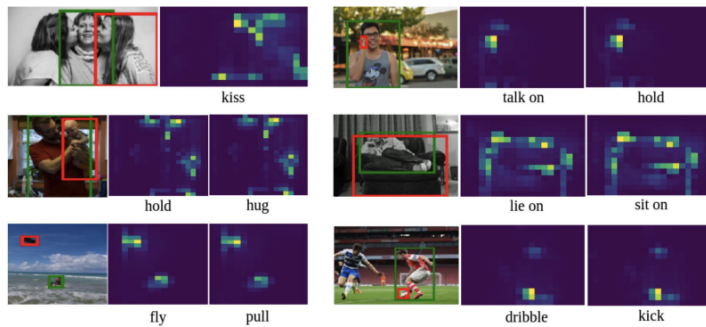


13.5.2 Object detection

The encoder (represent each pixel as a word) can separate individual instances, e.g. focusing on a few points in the image:



Different interaction categories have different patterns



14 Introduction to Reinforcement Learning

So far we have seen networks which are trained using supervised learning methods. Reinforcement networks are capable of learning a series of actions to train an agent **without direct supervision**. Such networks still require a goal to guide them, and this comes in the form of a "reward".

Reinforcement nets undertake a selection of actions, sometimes perhaps randomly. A reward is computed based on whether the action brought the system closer to achieving its goal.

After a number of such exploration phases, backpropagation (or similar optimisation) is used to update the network weights. Hence the network is trained such that given a current world state it recognises what to do next to move the world state in a direction consistent with achieving its overall goal.

Deep Recurrent Networks in combination with Deep Reinforcement learning is often used to build systems that learn

to undertake complex tasks without supervision, such as robotic grasping and manipulation.

Note: The environment and interpreter often get lumped together as a single unit.

14.1 Autonomous Driving example

Input: A single monocular image at each timestep

The model: DCNN

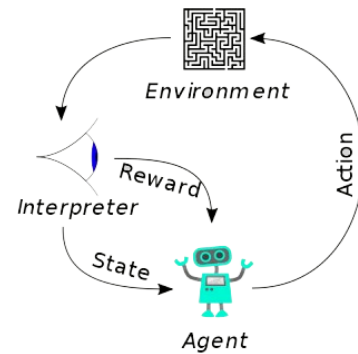
Output: steering angle and speed

Reward: distance travelled before human intervention (e.g. stop/slow down the car, override the steering). The model learns to maximise the reward, i.e. minimise/avoid human intervention.

14.2 Deep Reinforcement Learning (DRL)

The typical framing of a Reinforcement Learning scenario:

- An agent takes action in an environment
- An interpreter decides on a reward based on the outcome of the action
- And generates a representation of the environment state
- The reward and state are fed back into the agent

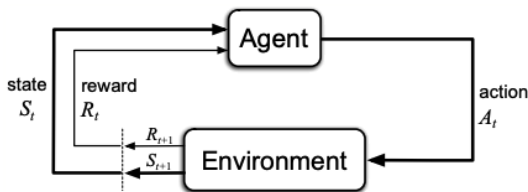


14.2.1 Core definitions

- **Agent:** An agent takes actions.
- **Action(A):** A is the set of all possible moves the agent can make
- **Discount factor (γ):** The discount factor is multiplied by future rewards as discovered to bias future rewards to be worth less than immediate rewards.
- **Environment:** The environment takes the agent's current state (i.e. environment and agent state) and action as input, and returns as output the agent's reward and its next state.
- **State (S):** A state is a concrete and immediate situation in which the agent finds itself. The current situation returned by the environment, or any future situation.
- **Reward (R):** A reward is the feedback by which we measure the success or failure of an agents actions. An agent sends output in the form of actions to the environment. the environment returns the agent's new state (which resulted from acting on the previous state) as well as rewards, if there are any.
- **Policy (π):** The policy is the strategy that the agent employs to determine the next action based on the current state. It maps states to actions, the actions that promise the highest reward.
- **Utility (U):** The expected long-term return with discount, as opposed to the short term reward R. $U\pi(s)$ is defined as the expected long term return of the current state under policy π .

We discount rewards, or lower their estimated utility, the further into the future they occur.

- **Q-value or action-value (Q):** Q-value is similar to Utility, except that it takes an extra parameter, the current action a .
 $Q\pi(s,a)$ refers to the long term returns of the current state s , taking action policy π .
 Q maps state-action pairs to rewards. Note the difference between Q and policy.
- **Trajectory:** A sequence of states and actions that influence those states.



14.2.2 Core Algorithm

- Environments are functions that transform an action taken in to the current state into the next state and a reward.
- Agents are functions that transform the new state and reward into the next action.
- We know the agent's function, but we cannot know the function of the environment. It is a black-box where we only see the inputs and outputs.
- In the feedback loop, the subscripts denote the time steps t and $t+1$, each of which refers to different states: the state at moment t , and the state at moment $t+1$.
- Unlike other forms of machine learning (such as supervised and unsupervised learning) reinforcement learning can only be thought about sequentially in terms of state-action pairs that occur one after the other.
- Reinforcement learning judges actions by the results they produce.
- It is goal-oriented, and its aim to learn sequences of actions that will lead an agent to achieve its goal, or maximise its objective function.

Evaluating Reward An example of a reward function

$$\sum_{t=0}^{t=\infty} \gamma^t R(x(t), a(t))$$

Sum reward function R over t time steps. This objective function calculates the total reward we obtain by running through a window of t time steps, for a finite number of steps t .

- x is the state at a given time step
- a is the action taken in that state
- R is the reward function for x and a
- γ is the discount function, to bias rewards from more immediate actions over future actions, where $\gamma \leq 1$.

Why γ ?

But this long run is a misleading guide to current affairs. **In the long run we are all dead.** Economists set themselves too easy, too useless a task if in tempestuous seasons they can only tell us that when the storm is long past the ocean is flat again.
 (John Maynard Keynes)

14.2.3 Action Selection

Reinforcement learning is the process of running the agent through sequences of state-action pairs.

- Observing the rewards that result
- Adapting the predictions of the Q function to those rewards until it accurately predicts the best path for the agent to take
- That prediction is known as a policy π

Reinforcement learning is an attempt to model a complex probability distribution of rewards in relation to a very large number of state-action pairs. Reinforcement learning is iterative. It doesn't begin by knowing which rewards state-action pairs will produce good rewards, it learns those relations by running through states again and again.

The goal of reinforcement learning is to pick the best-known action for any given state, this means the actions have to be ranked and assigned values relative to one another. Since those actions are state-dependent, we are gauging the value of state action pairs, i.e. the reward resulting from an action taken in the context of a certain state.

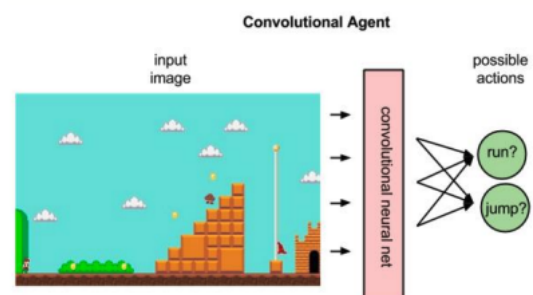
We map state-action pairs to the values we expect them to produce by means of the **Q function**. The Q function takes as its input an agent's state and action, and maps them to probable rewards.

14.2.4 Deep Implementations

Neural networks are the agent that **learns to map state-action pairs to rewards**. Like all neural networks, their learning consists of finding the right coefficients, or weights, by iteratively adjusting those weights along gradients that promise less error.

Convolution networks can also be used to **recognise an environments state**: e.g. the game environment that Mario is on, or the terrain before a drone, i.e. they perform their typical task of image recognition.

In reinforcement learning, given an image that represents a state, **a convolutional net can rank the actions possible to perform** in that state: For example, it might predict that running right will return 5 points, jumping 7, and running left none.



14.2.5 (Classical) Core Algorithm (Cont.)

- $a = \pi(S)$ where π is a policy that maps a state to an action
- Q maps state-action pairs to the highest combination of immediate reward, including all future rewards that might be harvested by later actions:

$$Q(s_t, a_t) \leftarrow (1 - \alpha) \cdot Q(s_t, a_t) + \alpha \cdot \left(\underbrace{r_t}_{\text{reward}} + \underbrace{\gamma}_{\text{discount factor}} \cdot \underbrace{\max_a Q(s_{t+1}, a)}_{\text{estimate of optimal future value}} \right)$$

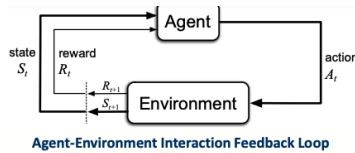
learned value

- State-action pairs mapped to the values we expect them to produce with the Q function.

- Q function inputs an agent's state and action, and maps them to probable rewards.
- Reinforcement learning is the process of running the agent through sequences of state-action pairs, observing the rewards that result, and adapting the predictions of the Q function to those rewards until it accurately predicts the best path for the agent to take.
- That prediction is known as a policy π
- The Q function operates over a time window to take both immediate and delayed rewards (blended by the learning rate α) from future actions - γ is a discount factor, to suppress reward from distant future actions.
- Having assigned values to the expected rewards, the Q function simply selects the state-action pair with the highest so called Q value:

$$Q(s_t, a_t) \leftarrow (1 - \alpha) \cdot Q(s_t, a_t) + \underbrace{\alpha}_{\text{learning rate}} \cdot \underbrace{\left(R_t + \gamma \cdot \max_a Q(s_{t+1}, a) \right)}_{\text{learned value}}$$

- The neural network coefficients may be initialised stochastically, or randomly.
- Using feedback from the environment, i.e. by applying an action, the neural net can use the difference between its expected reward and the ground-truth reward to adjust its weights and improve its interpretation of state-action pairs using optimisation, somewhat like backpropagation.

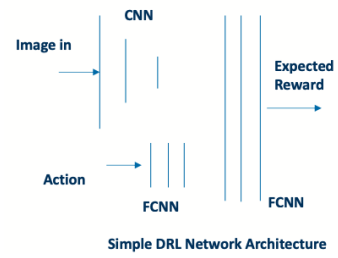


- The above feedback loop is analogous to the backpropagation of error in supervised learning.
- Reinforcement learning relies on the environment to send it a scalar number, i.e. reward, in response to each new action.
- The rewards returned by the environment can be varied, delayed or affected by unknown variables, introducing noise to the feedback loop.
- This leads us to a more complete expression of the Q function, which takes into account not only the immediate rewards produced by an action, but also the delayed rewards that may be returned several times steps deeper in the sequence.
- The Q function is recursive: calling the Q function on a given state-action pair requires us to call a nested Q function to predict the value of the next state, which in turn depends on the Q function of the state after that, and so forth...

14.2.6 Implementation Issues

Actual implementations vary widely depending on the context of the task and application scenario. Typically a DRL might spend a number of timesteps exploring followed by learning, i.e. network update.

Initially this exploration would be based on undertaking actions essentially either driven randomly, or based on initially little state-action-reward knowledge. However, the initial exploration is followed by updates of the networks which learn to rank (state, action) pairs. A simple reward predictor network is shown:



14.3 Deep Learning for Robots

Learning from large scale interaction (by Levine et al, Google research)

"Observing the behaviour of the robot after over 800,000 grasp attempts, which is equivalent to about 3000 robot-hours of practice, we can see the beginnings of intelligent reactive behaviours."

"The robot observes its own gripper and corrects its motions in real time. It also exhibits interesting pre-grasp behaviours, like isolating a single object from a group. All of these behaviours emerged naturally from learning, rather than being programmed into the system."

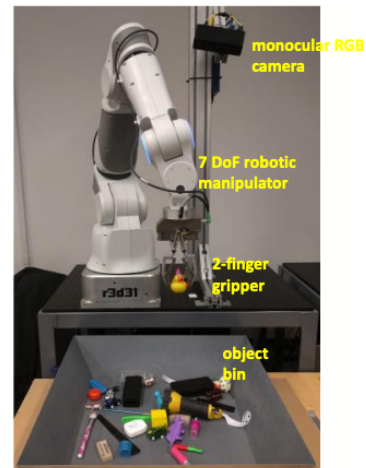
14.4 Learning Hand-Eye Coordination

CNN grasp serving approach: Entirely data driven - does not rely on any human annotation either at training or test time, contrasts to prior methods based on defined grasp points.

Method consists of two components: A grasp success predictor, which uses a deep convolution neural network (CNN) to determine how likely a given motion is to produce a successful grasp, and a continuous servoing mechanism that uses the CNN to continuously update the robot's motor commands. By continuously choosing the best predicted path to a successful grasp, the servoing mechanism provides the robot with

- Fast feedback to perturbations
- Ability to deal with object motion
- Robustness to inaccurate actuation (by correcting actuation mistakes iteratively)

The main contributions of this work are: A method for learning continuous visual servoing for robotic grasping from monocular cameras, a novel convolutional neural network architecture for learning to predict the outcome of a grasp attempt, and a large-scale data collection framework for robotic grasp.

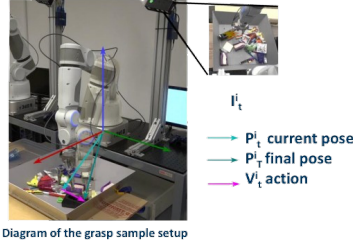


14.4.1 Grasp Success Prediction

Grasp prediction network $g(l_t; v_t)$ is trained to predict whether a given task-space motion v_t will result in a suc-

successful grasp, based on the current camera observation l_t . In order to make accurate predictions, $g(l_t; v_t)$ must be able to

- Parse the current camera image
- Locate the gripper
- Determine whether the gripper according to v_t will put it in a position where closing the fingers will pick up an object



Each grasp i consists of T time steps, each with time step corresponding to an image I_t^i and pose P_t^i . The final dataset contains samples $(I_t^i; p_T^i - p_t^i; lb_i)$ that consist of the

- Image, I_t^i
- A vector from the current pose to the final pose $p_T^i - p_t^i$.
- The grasp success label, lb_i

As mentioned, each grasp consists of T time steps, at each time step, the robot records the current image I_t^i and the current pose p_t^i , and then chooses a direction along which to move the gripper. At the time step T , the robot closes the gripper and evaluates the success of the grasp, producing a label lb_i . Each grasp attempt results in T training samples, given by $(I_t^i; p_T^i - p_t^i; lb_i)$

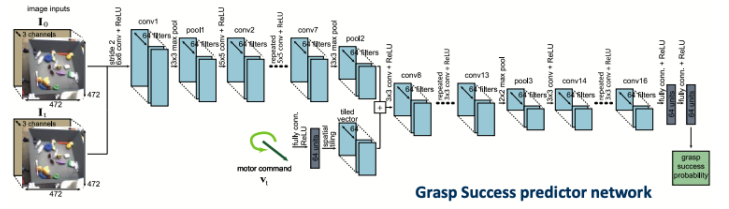
Each sample includes the image observed at that time step, the vector from the current pose to the one that is eventually reached, and the success of the entire grasp. **This procedure trains the network to predict whether moving a gripper along a given vector and then grasping will produce a successful grasp.**

Note that this differs from the standard reinforcement learning setting, where the prediction is based on the current state and motor command, which in this case is given by (I_t^i, p_t^i, v_t^i)

14.4.2 CNN Grasp Predictor Architecture

Recall CNN grasp predictor architecture $g(l_t; v_t)$

- Input image I_t & pre-grasp image I_0 (no grippers in FoV), fed into a 6x6 convolution with stride 2
- Followed by 3x3 max-pooling and 6 5x5 convolutions.
- Followed by a 3x3 max-pooling layer.
- Motor command v_t is processed by one fully connected layer pointwise added to each point in the response map of pool2.
- Result is then processed by 6, 3x3 convolutions, 2x2 max-pooling, 3 more 3x3 convolutions, and 2 fully connected layers with 64 units.
- Network outputs the probability of a successful grasp through a sigmoid.
- Each convolution is followed by batch normalisation.
- Once trained the network $g(I_t : v_t)$ can predict the probability of success of a given motor command, independently of the exact camera pose



14.4.3 CNN Continuous Servoing Architecture

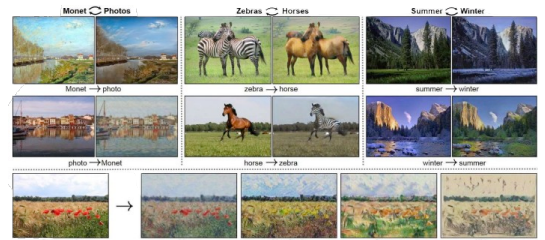
- A servoing mechanism $f(I_t)$ uses the prediction network to control the robot continuously to servo the gripper to a successful grasp
- The servoing mechanism performs inference using the grasp predictor $g(I_t : v_t)$, in order to determine the motor command v_t given an angle I_t
- Rather than randomly sampling v_t commands and feeding these to $g(I_t : v_t)$, run an optimiser to determine the best available command v_t^* and thus evaluate $f(I_t)$
- Use the predicted grasp success $p(I = 1)$ produced by the network to inform a heuristic for raising and lowering the gripper, as well as to choose when to stop moving and attempt a grasp, i.e. when the predicted grasp success label becomes true, use a preprogrammed function to cause the gripper to close on the target object

15 Introduction to Generative Models

They are used for tasks like image synthesis, to generate new images, or for image-to-image translation for style transfer, image restoration, etc. Generative models are different from what we have discussed so far, i.e. image analysis. They learn the underlying distribution of the data then create some new content from the learned model. e.g. GAN, VAE, Diffusion Models.

15.1 Generative Adversarial Networks (GANs)

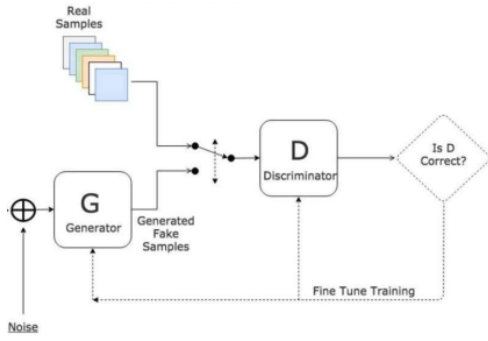
Image-to-image transfer: GANs are capable of learning highly sophisticated image mapping.



15.1.1 GAN Characteristics

The first model is a **generator**: Generates new data of the desired type, and could be thought of as a forger creating fake as opposed to real (natural) images.

The second model is a **discriminator**, goal is to recognise if input is 'real' → belongs to the original dataset, or if it is 'fake' → produced by the generator. Analogous to an expert who attempts to establish fidelity or otherwise.



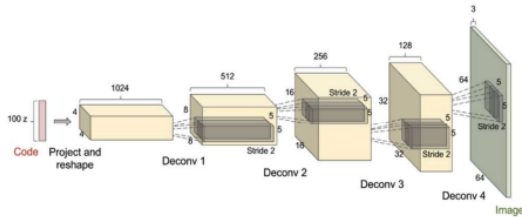
A neural network $G(z, \theta_1)$ is used to model the generator. Maps input noise variables z to the desired data space x (say images). A specific image latent space activation is combined with noise to generate a plausible variation of the input image.

A second neural network $D(x, \theta_2)$ models the discriminator. Outputs the probability $[0..1]$ that the data came from the real dataset. θ_i above represents the weights, or parameters that define each neural network.

15.1.2 Mathematically Modelling a GAN

When discriminator is trained to correctly classify the input data as either real or fake, its weights are updated as to maximise the probability that any real data input x is classified as belonging to the real dataset, while minimising the probability that any fake image is classified as belonging to the real dataset.

The loss/error function used maximises the function $D(x)$ and it also minimises $D(G(z))$



Generator is trained to fool the discriminator by generating data as realistically as possible. This the generators weights are optimised to maximise the probability that any fake image is classified as belonging to the real dataset. The loss/error function used for this network maximises $D(g(z))$. Logarithm of the probability (e.g. $\log D(\dots)$) is used in the loss functions. A log loss heavily penalises classifiers that are confident about an incorrect classification.

After several steps of training, if the generator and discriminator have enough capacity, and if the networks can approximate the objective functions:

- A point will then be reached at which neither can improve anymore
- Generator now generates realistic synthetic data
- Discriminator is unable to differentiate between the two types of input

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

Minimax game with value function $V(G, D)$

During training both the discriminator and generator are trying to optimise opposite loss functions. They can be thought of as two agents playing a minimax game with value function $V(D, G)$. In this minimax game, generator is trying to maximise its probability of having its outputs recognised as real, and discriminator is trying to minimise this same value.

Image Code Latent Space Arithmetic Deep Convolutional Generative Adversarial Networks (DCGANs) demonstrated that GANs can learn a distributed representation that disentangles the concept of gender from the concept of wearing glasses.

- Begin with the representation of a man with glasses
- The subtract the representing of a man without glasses
- And finally add the representing of a woman without glasses
- We obtain the vector representing the concept of a woman with glasses

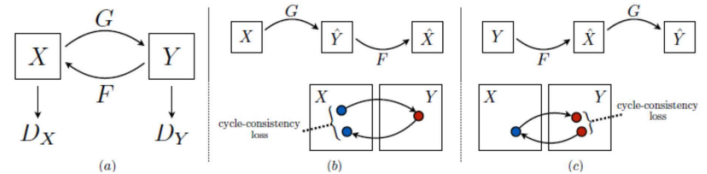
The generative model correctly decodes all of these representations vectors to images that may be recognised as belonging to the correct class.

15.2 CycleGANs

A major issue with implementing GANs is the requirement for paired training data. When mapping from domain A to domain B, examples for the same instance of each domain are required in training.

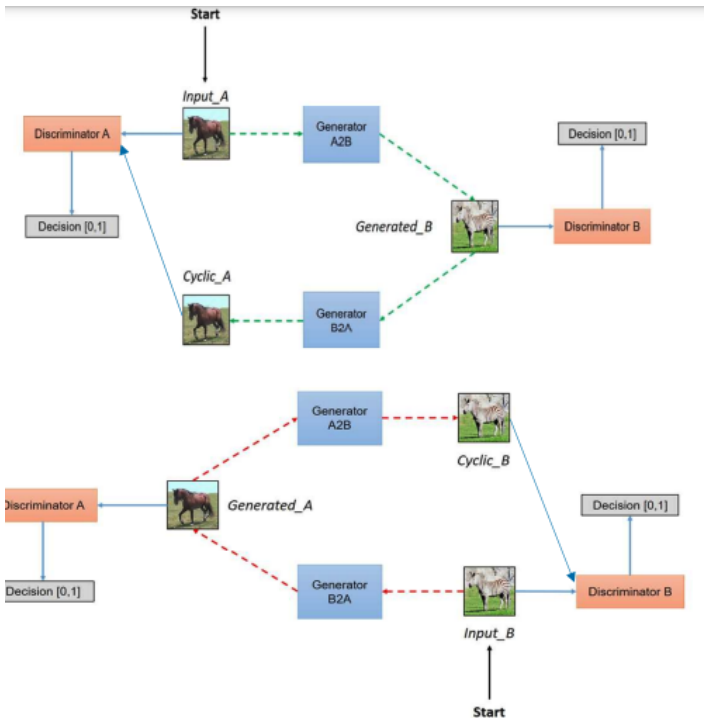
CycleGAN removes the need for paired training data - it learns the underlying image distribution and maps between these.

CycleGAN comprises two generators, consisting of Encoder and Decoder, and two discriminators, each implemented as a PatchGAN, i.e. to evaluate if each local patch is real or fake.



CycleGAN model contains two mapping functions $G : X \rightarrow Y$ and $F : Y \rightarrow X$, and associated adversarial discriminators D_y and D_x . D_y encourages G to translate X into outputs indistinguishable from domain Y , and vice versa for D_x and F . Two cycle consistency losses.

- To minimise the difference after translating into another domain and then back to the original one
- Forward cycle consistency loss: $x \rightarrow G(x) \rightarrow F(G(x)) \approx x$
- Backward cycle consistency loss: $x \rightarrow G(y) \rightarrow F(G(y)) \approx y$



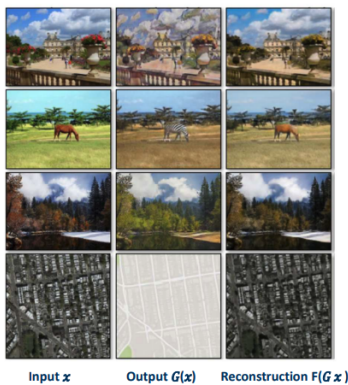
1. Feed an input image from domain Do_A to our first generator $Generator_{A \rightarrow B}$ To transform a given image from Do_A to Do_B
2. Feed the newly generated image to another generator $Generator_{B \rightarrow A}$
3. This output image must be close to the original input image to define a meaningful mapping that is absent the unpaired dataset. We need cycle consistency losses.

Two inputs are fed into each discriminator: one is the original image corresponding to that domain, the other is the generated image via a generator, **or** can be formulated to classify a **single** image as real or fake.

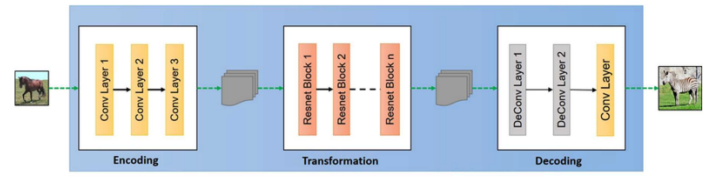
The job of each discriminator is to distinguish between them - defy the adversary (in this case generator) and reject images generated by it. Each generator would like to make sure their generated image gets accepted by the discriminator - to make generated images very close to the original ones.

The input images x , output images $G(x)$, and the reconstructed images $F(G(x))$ from various experiments. From top to bottom:

- photo $\leftrightarrow$ Cezanne,
- horses $\leftrightarrow$ zebras
- winter $\leftrightarrow$ summer
- Yosemite, ariel photos $\leftrightarrow$ Google maps



A generator's encoding, transformation, and decoding pipeline looks something like;



Generators encoding:

- Input: Image from domain Do_A of size $[256,256,3]$
- Output: $[64,64,256]$

Generator's transformation

- Combining different nearby features of an image, and
- Making a decision about how to transform that feature vector/encoding (output of an image from Do_A to that of Do_B)
- In CycleGAN, 6 layers of **Resnet** blocks were used.
- Final output: $[64,64,256]$, seen as the feature vector for an image in domain Do_B
- The transformation **resnet** block consists of two convolution layers, with a residue of input added to the output to ensure properties of input of previous layers are available for layer layers as well.

This is done so that the output does not deviate much from the original output, otherwise the characteristics of the original image will not be retained, and results will be very abrupt

- This is to retain the characteristics of the original input like the size and the shape of the object. Residual networks are very suitable for these kinds of transformations

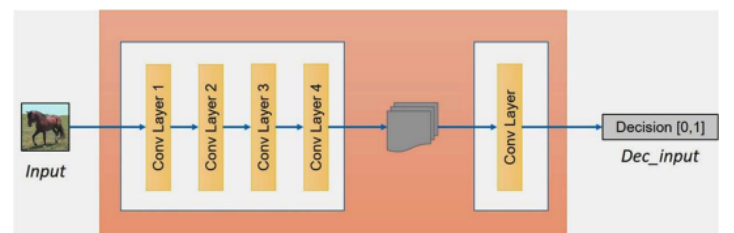
Generator's Decoding

- We get another feature vector from transformation with size $[64,64,256]$
- Decoding step is the exact opposite of the encoding step, to rebuild the low-level features back from the feature vector
- This is done by applying a deconvolution or transpose convolution layer
- We will convert this low-level feature to an image in domain Do_B
- Finally we have the generated image of size $[256,256,3]$

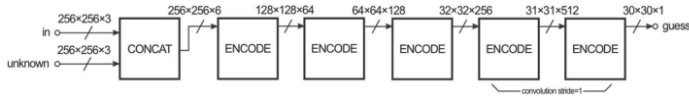
15.2.1 The Discriminator

Single image, no real input for comparison

The discriminator is simply a convolutional neural network in our case. First, we will extract the features from the image. Next, deciding whether these features belong to that particular category or not. For that we will add a final convolution layer that produces a 1 dimensional output



Note: Only a single image to be evaluated as real or fake is input to the discriminator used in these experiments. Though some formulations can use two inputs, as shown before, comprising the real and generated images under comparison



15.2.2 Training

To train this network, there are two steps: training the discriminator, and training the generator. To train the discriminator, first the generator generates an output image. The generator looks at the output and produces its guess about how realistic it looks.

The weights of the discriminator are then adjusted based on the classification error of the output. The generator's weights are then adjusted based on the output of the discriminator.

Combining the cycle consistency loss gives us: $L_{cyc}(G, F)$, and combining with the adversarial losses on domains:

$$X(L_{GAN}(G, D_Y, X, Y)) \\ Y(L_{GAN}(F, D_X, Y, X))$$

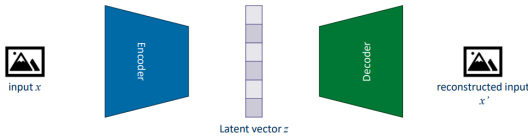
Yielding the overall CycleGAN objective expressed as

$$L(G, F, D_X, D_Y) = L_{GAN}(G, D_Y, X, Y) + L_{GAN}(F, D_X, Y, X) + \lambda L_{cyc}(G, F)$$

Where D_Y is the associated adversarial discriminator that encourages G to translate X into outputs indistinguishable from domain Y , and vice versa for D_X . λ is a constant that controls the relative importance of the two objective functions G and F in the cycle consistency loss

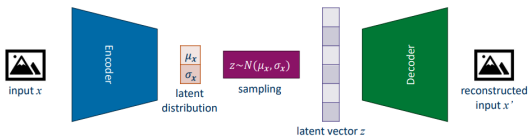
15.3 Variational Autoencoder(VAE)

Autoencoders (AEs) learn a latent presentation to enable a 1-to-1 mapping between the input and the output.



Trained by minimising reconstruction loss, e.g. $\|x - x'\|_2$ so we can generate new output (image) by giving 'random' latent representation to the decoder.

The VAE first learns the distribution of the latent space, i.e. maps input to multivariate normal distribution



Trained by minimising 1) reconstruction loss, and 2) Kullback-Leibler (KL) divergence to measure distance loss between $N(\mu_x, \sigma_x)$ and $N(0, 1)$

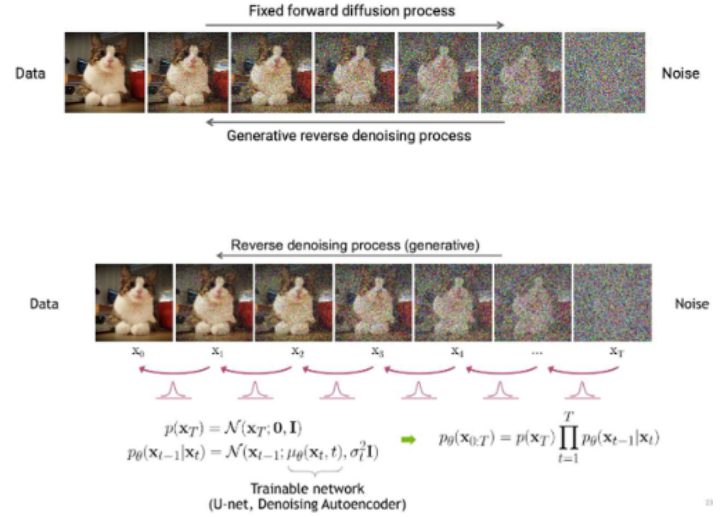
Then, latent representation can be sampled from such distribution and be used as input of the decoder - to generate new images at inference.

Conditional VAE - add conditions (such as text, class label, etc) to control image generation, the condition may need to be encoded first.

15.4 Diffusion (probabilistic) Models

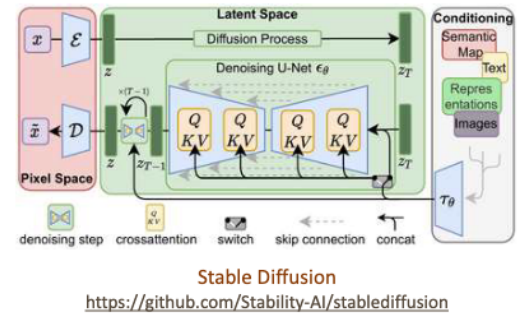
2 Processes:

- Forward diffusion process - to gradually map input data to noise
- Reverse diffusion process - to generate data from pure random noise through iterative denoising, using a trained model



15.5 More control on Generative Models

We gain more control by providing conditions as input, e.g. input, text, and class labels.



Stable Diffusion
<https://github.com/Stability-AI/stablediffusion>

Examples are CGAN (conditional GAN), CVAE(conditional VAE), and stable diffusion, and a lot more!